

Configuration API

At a business level, a tenant is defined by its Metadata Configuration (also known as L3) and its UI Configuration. The methodology usually includes performing a GET Configuration (with no inheritance) to retrieve the current L3 configuration, using a JSON editor to modify the configuration, then using the Set Configuration API to PUT the resulting L3 back to your tenant. The two most important and often used API calls in this section are GET Configuration and Set Configuration. Add and Remove Source endpoints are seldom used. Sources are usually defined in the L3 during configuration using the methodology above.

When a tenant's Metadata (L3) configuration needs to be updated through the PUT API, you must:

- Run the [Tenant configuration Validation API](#) before you post the new L3 configuration into the tenant.
- Scan the metadata configuration for errors or any violation of best practices. If any errors are found, then these errors should be fixed before posting the business configuration to the tenant.

Run the Validate Tenant Configuration API to validate the tenant configuration:

```
GET {{envUri}}/reltio/tenants/{{tenantId}}/_validateTenantConfiguration
```

Replace {{envUri}} with the base URL of your target environment and {{tenantId}} with the ID of the target tenant.

Note: It is recommended to perform a **GET** with Inheritance to understand the consolidated configuration of all the layers (L1, L2, L3). But, when editing the configuration, it is important to **Get** and **Set** only the L3 layer of the configuration. To do this you must make another **GET** request using the **_noinheritance** option. This retrieves only the L3 layer which is the layer tightly bound to your tenant. Once you have modified it to your liking then you should use the **Set** Configuration API to **PUT** only that L3 layer back to your tenant. **Never apply a consolidated configuration to the tenant.**

For additional information about configuration validation errors, refer to [L3 validation errors](#).

For additional information about configuration concepts, refer to [Tenant data configuration](#).

In addition to retrieving or replacing the full configuration, you can access, modify, or delete individual configuration types and sub-resources. To manage only one object type for example, an entity or relation type, see [Manage configuration by object type](#). To handle sub-resources such as **cleanseConfig**, or survivorship groups, see [Manage configuration sub-resources](#).

The Configuration API allows you to manage your tenant's metadata configuration. The Reltio Console supports management of selected configuration settings, while the complete metadata configuration can be modified using only the Configuration API.

Object Structure

This section describes the first-level properties of the Configuration resource.



Property name	Required	Description	Type
		Path that is used to reach configuration. It is always	

description	No	Details for a configuration.	String
label	No	Readable name for a model.	String
schemaVersion	Yes	Version of Reltio API Configuration schema used to describe objects.	String
entityTypes	Yes	A collection of Entity Types defined for a tenant.	Array of Entity Types
roles	No	A collection of Role Types defined for a tenant.	Array of Roles
groupTypes	No	A collection of Group Types defined for a tenant.	Array of Group Types
relationTypes	No	A collection of Relation Types defined for a tenant.	Array of Relation Types
graphTypes	No	Define the Graph Types in a tenant configuration.	Array of Graph Types
interactionTypes	No	A collection of Interaction Types defined for a tenant.	Array of Interaction Types
sources	No	A collection of Interaction Types defined for a tenant.	Array of Sources

Configuration JSON Example

```
{
  "URI": "configuration",
  "label": "Test Business Model",
  "description": "Test Business Model Configuration-used to describe Configuration API format",
  "schemaVersion": "1",
  "entityTypes": [
    {
      "URI": "configuration/entityTypes/Individual",
      "label": "Individual",
      "id": "3",
      "description": "These entity types for representing people",
      "typeColor": "#00FF00",
      "typeIcon": "entityType/Individual.png",
      "typeGraphIcon": "entityType/IndividualSmall.png",
      "typeImage": "defaultImage/no-photo.png",
      "dataLabelPattern": "(configuration/entityTypes/Individual/attributes/FirstName)",
      "entitySmartLogic": "Person",
      "entityTypeRoleURIs": [
        "configuration/roles/Client",
        "configuration/roles/Advisor"
      ],
    },
  ],
  "attributes": [
    {
      "URI": "configuration/entityTypes/Individual/attributes/FirstName",
      "name": "FirstName",
      "id": "4",
```

```
        "label": "First Name",
        "type": "String"
    },
    {
        "URI": "configuration/entityTypes/Individual/attributes/Gender",
        "name": "Gender",
        "label": "Gender",
        "id": "5",
        "type": "String",
        "values": [
            "Male",
            "Female",
            "Unknown"
        ]
    },
    {
        "URI": "configuration/entityTypes/Individual/attributes/ImageLinks",
        "name": "ImageLinks",
        "id": "6",
        "label": "Image Links",
        "type": "Image URL"
    },
    {
        "URI": "configuration/entityTypes/Individual/attributes/Address",
        "name": "Address",
        "label": "Address",
        "id": "9",
        "type": "Reference",
        "referencedEntityTypeURI": "configuration/entityTypes/Location",
        "relationshipTypeURI": "configuration/relationshipTypes/HasAddress",
        "relationshipLabelPattern": "{configuration/relationshipTypes/HasAddress/attributes/AddressType}",
        "referecedAttributeURIs": [
            "configuration/relationshipTypes/HasAddress/attributes/AddressType",
            "configuration/entityTypes/Location/attributes/City"
        ]
    }
    ],
    "businessCardAttributeURIs": [
        "configuration/entityTypes/Individual/attributes/FirstName",
        "configuration/entityTypes/Individual/attributes/Gender"
    ],
    "imageAttributeURIs": [
        "configuration/entityTypes/Individual/attributes/ImageLinks"
    ]
},
{
    "URI": "configuration/entityTypes/Location",
    ...
}
],
"roles": [
    {
        "URI": "configuration/roles/Client",
        "label": "Client",
        "id": "1",
        "description": "Client role-highlight entities who are clients"
    },
    {
        "URI": "configuration/roles/Advisor",
        "label": "Advisor",
        "id": "2",
        "description": "Advisor is an employee of a company working with clients."
    }
],
"groupTypes": [
    {
        "URI": "configuration/groupTypes/XYZHHGroupType",
        "id": "16",
        "label": "XYZ Household",
        "type": "manual",
        "description": "Information about households loaded from XYZ system",
        "dataLabelPattern": "[primaryMember]({configuration/entityTypes/Individual/attributes/FirstName}) Household",
        "hasPrimaryMember": true,
        "attributes": [
            {
                "URI": "configuration/groupTypes/XYZHHGroupType/attributes/Notes",
                "name": "Notes",
                "id": "17",
                "label": "Notes",
                "type": "String"
            }
        ]
    },
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/groupElements/IndividualEntities",
        "id": "18",
        "entityTypeURIs": [
            "configuration/entityTypes/Individual"
        ]
    }
],
"memberTypes": [
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/memberTypes/Member",
        "id": "19",
        "label": "Member",
        "primaryMember": false
    },
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/memberTypes/PrimaryMember",
        "id": "20",
        "label": "Primary Member",
        "primaryMember": true
    }
]
```

```
        "label": "First Name",
        "type": "String"
    },
    {
        "URI": "configuration/entityTypes/Individual/attributes/ImageLinks",
        "name": "ImageLinks",
        "id": "6",
        "label": "Image Links",
        "type": "Image URL"
    },
    {
        "URI": "configuration/entityTypes/Individual/attributes/Address",
        "name": "Address",
        "label": "Address",
        "id": "9",
        "type": "Reference",
        "referencedEntityTypeURI": "configuration/entityTypes/Location",
        "relationshipTypeURI": "configuration/relationshipTypes/HasAddress",
        "relationshipLabelPattern": "{configuration/relationshipTypes/HasAddress/attributes/AddressType}",
        "referecedAttributeURIs": [
            "configuration/relationshipTypes/HasAddress/attributes/AddressType",
            "configuration/entityTypes/Location/attributes/City"
        ]
    }
    ],
    "businessCardAttributeURIs": [
        "configuration/entityTypes/Individual/attributes/FirstName",
        "configuration/entityTypes/Individual/attributes/Gender"
    ],
    "imageAttributeURIs": [
        "configuration/entityTypes/Individual/attributes/ImageLinks"
    ]
},
{
    "URI": "configuration/entityTypes/Location",
    ...
}
],
"roles": [
    {
        "URI": "configuration/roles/Client",
        "label": "Client",
        "id": "1",
        "description": "Client role-highlight entities who are clients"
    },
    {
        "URI": "configuration/roles/Advisor",
        "label": "Advisor",
        "id": "2",
        "description": "Advisor is an employee of a company working with clients."
    }
],
"groupTypes": [
    {
        "URI": "configuration/groupTypes/XYZHHGroupType",
        "id": "16",
        "label": "XYZ Household",
        "type": "manual",
        "description": "Information about households loaded from XYZ system",
        "dataLabelPattern": "[primaryMember]({configuration/entityTypes/Individual/attributes/FirstName}) Household",
        "hasPrimaryMember": true,
        "attributes": [
            {
                "URI": "configuration/groupTypes/XYZHHGroupType/attributes/Notes",
                "name": "Notes",
                "id": "17",
                "label": "Notes",
                "type": "String"
            }
        ]
    },
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/groupElements/IndividualEntities",
        "id": "18",
        "entityTypeURIs": [
            "configuration/entityTypes/Individual"
        ]
    }
],
"memberTypes": [
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/memberTypes/Member",
        "id": "19",
        "label": "Member",
        "primaryMember": false
    },
    {
        "URI": "configuration/groupTypes/XYZHHGroupType/memberTypes/PrimaryMember",
        "id": "20",
        "label": "Primary Member",
        "primaryMember": true
    }
]
```

```
]
}
],
"relationTypes":[
{

```

```
"typeColor": "#663344",
"implicit": true,
"direction": "undirected",
"attributes": [
{
"URI": "configuration/relationTypes/Spouse/attributes/WeddingPlace",
"name": "WeddingPlace",
"id": "26",
"label": "Wedding Place",
"type": "String"
}
],
"startObject": {
"URI": "configuration/relationTypes/Spouse/startObject",
"objectURI": "configuration/roles/Party"
},
"endObject": {
"URI": "configuration/relationTypes/Spouse/endObject",
"objectURI": "configuration/entityTypes/Location"
}
},
{
"URI": "configuration/relationTypes/HasAddress",
...
}
],
"interactionTypes": [
{
"URI": "configuration/interactionTypes/Meeting",
"id": "31",
"label": "Meeting",
"attributes": [
{
"URI": "configuration/interactionTypes/Meeting/attributes/Place",
"name": "Place",
"id": "32",
"label": "Place",
"type": "String"
},
{
"URI": "configuration/interactionTypes/Meeting/attributes/Notes",
"name": "Notes",
"id": "33",
"label": "Notes",
"type": "String"
}
]
},
{
"URI": "configuration/interactionTypes/Meeting/memberTypes/participants",
"name": "participants",
"id": "34",
"label": "participants",
"objectURI": "configuration/entityTypes/Individual",
"minOccurs": 1
},
{
"URI": "configuration/interactionTypes/Meeting/memberTypes/organizers",
"name": "organizers",
"id": "35",
"objectURI": "configuration/entityTypes/Individual",
"label": "organizers",
"minOccurs": 1
}
]
},
"graphTypes": [
{
"URI": "configuration/graphTypes/SpouseGraph",
"id": "37",
"graphStructure": "network",
"type": "logical",
"layout": "family",
"label": "Family Graph",
"relationshipTypeURIs": [
"configuration/relationshipTypes/SpouseGraph"
]
}
],
"sources": [
{
"URI": "configuration/sources/XYZ",
"id": "39",
"abbreviation": "XYZ",
"label": "MDM",
"icon": "source/xyz.png"
},
{
"URI": "configuration/sources/Reltio",
"id": "40",
"abbreviation": "Reltio",
"label": "Reltio source system",
"icon": "source/reltio.png"
}
]
```

```
}  
}
```

Use [Validation API](#) ^[API] to run full validation of a tenant's business configuration (metadata configuration) and return validation errors and/or warnings. Use the GET endpoint to validate the business configuration stored for the tenant. Use the POST endpoint to validate a business configuration JSON payload without saving it. This topic covers tenant-level business configuration validation only.

HTTP method and endpoint

Use the following HTTP methods and endpoint paths to validate a tenant business configuration.

Method	Endpoint	Description
GET	{{envUri}}/relio/api/{{tenantId}}/configuration/_validateTenantConfiguration?validationErrorLevel=<validation-level>	Validates the stored business configuration stored in the database for the tenant.
POST	{{envUri}}/relio/api/{{tenantId}}/configuration/_validation?validationErrorLevel=<validation-level>	Validates a configuration JSON payload provided in the request body without saving it.

The following table describes the endpoint path parameters.

Parameter	Type	Required	Description
tenantId	String	Yes	The unique identifier of the tenant. Specifies the tenant context for the request. Example: ce5627DYnQ6abcD

Query parameter

The following table describes the query parameter and its value.

Parameter	Type	Required	Description	Accepted values / Default	When to use
validationErrorLevel	String	Yes	Filters the severity level of validation findings returned in the response output. Replace <validation-level> with one of the accepted values — all , error and warning .	all	Use this value when you want the response to include both validation errors and warnings. This is the default value.
				warning	Use this value when you want to review only validation warnings.
				error	Use this value when you want to review only validation errors.

Request headers

The following request headers must be included.

Header	Value	Required
Authorization	Bearer <Access_Token>	Yes
Content-Type	application/json	Yes (POST only)

Request body

The following table describes the request body requirements for each HTTP method.

ReltioProductsDeveloper resourcesRolesApplicationsObjectives

GET	Not required	This operation does not require a request body.
POST	Required (JSON payload)	Send the business configuration that you want to validate in the request body as a JSON payload. The API validates the payload and returns validation findings without saving the configuration to the database.

Example request

Use the following example to see how a complete request is structured.

GET example

```
GET {{envUri}}/reltio/api/{{tenantId}}/configuration/_validateTenantConfiguration?validationErrorLevel=all
```

POST example

```
POST {{envUri}}/reltio/api/{{tenantId}}/configuration/_validation?validationErrorLevel=all

{ ...configuration payload to validate... }
```

Response body

The following table describes the fields returned in the response body.

The response groups results by configuration path (for example, **configuration/entityTypes/Organization**), and each configuration path includes a **warnings** array.

Field	Type	Description
warnings	array of object	The list of validation findings for the configuration path.
severity	string	The severity level of the finding. Example: Error .
errorMessage	string	A short message that describes the finding.
errorCode	integer	The numeric code for the finding. Example: 1801 .
errorDetailMessage	string	A detailed message that describes the finding and how to fix it.

Example response

The following example shows a successful response. The response format is the same for both the GET and POST endpoints.

```
{
  "configuration/entityTypes/Organization": {
    "warnings": [
      {
        "severity": "Error",
        "errorMessage": "No surrogate crosswalk defined for entityconfiguration/entityTypes/Organization",
        "errorCode": 1801,
        "errorDetailMessage": "No surrogate crosswalk defined for entityconfiguration/entityTypes/Organization, Please define surrogate crosswalk"
      }
    ]
  },
}
```

```
"configuration/entityTypes/Location": {
  "warnings": [
    {
      "severity": "Error",
      "errorMessage": "No surrogate crosswalk defined for entityconfiguration/entityTypes/Location",
    }
  ]
}
```

```
}
}
}
```

Add a Source System

You can add the source system by posting a request. If the request is successful, then it returns a JSON that displays the attributes of the added source type.

Request

```
POST {TenantURL}/configuration/sources
```

Parameters

Parameter	Name	Required	Description
Headers	Content-Type	Yes	The value should be "Content-Type: application/json".
Body		Yes	JSON object with Source type specification in Reltio Business Model format.

Response

If a source type is created successfully, then the response will contain a JSON with this source type.

Example Request

```
POST {TenantURL}/configuration/sources
Body:{
  "uri": "configuration/sources/GAR",
  "label": "GAR",
  "abbreviation": "GAR",
  "icon": "images/source/twitter.png"
}
```

Example Response

```
POST {TenantURL}/configuration/sources
Body:
{
  "uri": "configuration/sources/GAR",
  "label": "GAR",
  "abbreviation": "GAR",
  "icon": "images/source/twitter.png"
}
```

Add an entity type

You can add entity type as part of overall L3 configuration. For more details, see topic [Configuring Survivorship Rules in the L3](#) .

Entity type metadata

This table lists the metadata properties for an entity type that you can configure using the [Set Configuration](#) or [Data Modeler at a glance](#).

Property	Required	Description	Type
URI	Yes	Path that the software uses to reach the entity type. Format:	String, URI

ReltioProductsDeveloper resourcesRolesApplicationsObjectives

id	Yes-for response JSON	Internal, system-generated ID. Used internally to get the difference between two versions of configurations when configuration is updated.	String
label	Yes	Readable name of the entity type. Example: Individual	String
description	No	Description for the entity type.	String
extendsTypeURI	No	URI of an entity type that is extended by this entity type. Example: configuration/roles/Party	String, URI
typeColor	No	Color that the software uses is used to visualize this entity type. For example, in Graph View, it's the color of a node. Example: #00FF00	String, hex color
typeIcon	No	Icon used in UI facets when representing the entity type.	String, URL
typeGraphIcon	No	Icon used in UI graphs when representing the entity type.	String, URL
typeImage	No	Default profile image.	String, URL
dataLabelPattern	No	Pattern that the software uses to build an entity object label. Can include static text, as well as patterns-attributes in curly brackets. You can use square brackets to define conditional label patterns. If an attribute used inside the set of brackets doesn't exist, then all content inside the brackets are skipped.  Note: If dataLabelPattern is not specified, label is used. Example: {FirstName} {MiddleName} {LastName}, {Credentials}	String
secondaryLabelPattern	No	Pattern that the software uses to build an entity object secondary label. Can include static text, as well as patterns-attributes in curly brackets.  Note: You can use comma-separated string built from business card attributes. In this case, use {businessCardAttributes} as a string value. Examples: {businessCardAttributes} OR {Address}.	
dataTooltipPattern	No	Pattern that the software uses to build the tooltip for an entity object. Can include static text, as well as patterns-attributes in curly brackets.  Note: If dataTooltipPattern is not specified, a label is used. Example: Tooltip: {configuration/entityTypes/Individual/attributes/FullName}	String
entitySmartLogic	No	Identifies when the entity type has smart logic. Possible values: <i>Person, Organization, Location</i> .	String
entityTypeRoleURIs	No	Array of URIs for a role that can be used in combination with this entity type. For example, an <i>Individual</i> entity object can be a client, while a <i>Location</i> object cannot be. For more details, see Entity Roles. Example: ["configuration/roles/Client", "configuration/roles/Advisor"]	Array of role URIs

Property	Required	Description	Type
attributes	No	Attributes that are defined as standard for this entity type. Attributes	JSON object
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			
		Example: { "URI": "configuration/entityTypes/Individual/attributes/FirstName", "name": "FirstName", "id": "4", "label": "First Name", "type": "String"} 	
defaultFacetedAttributes	No	Array of attributes that must be faceted for that entity. Example: "defaultFacetedAttributes": ["configuration/entityTypes/Plan/attributes/Type", "configuration/entityTypes/Plan/attributes/PaymentType"] 	Array of attributes
businessCardAttributeURIs	No	Attributes that are returned as search result items and displayed in the Profile band; that is, the top item on the Profile page with profile icon and label. There are three groups of attributes: - Label: Attributes from labelPattern. - Secondary label: Attributes from secondaryLabelPattern. - Business card: Attribute from businessCardAttrirbuteURIs. Can be used in place of secondary labels. The business card attributes are an array of URIs for attributes that must be used in a business card. These attributes follow in the order of priority and the order is preserved. Example: <pre>"businessCardAttributeURIs": ["configuration/entityTypes/Individual/attributes/FirstName", "configuration/entityTypes/Individual/attributes/Gender"],</pre> For details, see Attributes Configuration	Array of attributes URIs)
imageAttributeURIs	No	Array of URIs for attributes that contains entity object images. The system considers the first array element as the default <i>bucket</i> for profiles. Example: ["configuration/entityTypes/Individual/attributes/ImageLinks"] For details, see Attributes Configuration .	Array of attributes URIs
matchGroups	No	Array of match groups specification. For details, see Match Rule Configuration - Step by Step .	Array
cleanse	No	Enables you to define a set of data cleanse rules based on an entity type by specifying: - Cleanse service inputMapping: Defines which attributes must be cleansed to provide attributes URI outputMapping: Service output mappings - mapping of cleanse output to entity type attributes	JSONArray

Property	Required	Description	Type
cleanselInputObjectBeforeOverwrite	No	Enables (value is set to true) or disables (value is set to false) the	Boolean
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			

property is set to **false**, cleanse is skipped.
Sample entity request

```
[
  ...
  {
    "type": "configuration/entityTypes/Location",
    "tags": [
      "sport",
      "movies"
    ],
    "attributes": {
      "AddressLine1": [
        {
          "value": "00836 W Wewlington Ave F1 5"
        }
      ],
      "AddressLine2": [
        {
          "value": "Chicago EYE"
        }
      ],
      "StateProvince": [
        {
          "value": "IL"
        }
      ],
      "City": [
        {
          "value": "Chicago"
        }
      ],
      "Country": [
        {
          "value": "USA"
        }
      ],
      "Zip": [
        {
          "value": {
            "Zip5": {
              "value": "60657"
            }
          }
        }
      ]
    },
    "crosswalks": [
      {
        "type": "configuration/sources/FB",
        "value": "e1"
      }
    ]
  }
]
```

Entity after it is updated

```
[
  ...
  {
    "type": "configuration/entityTypes/Location",
    "tags": [
      "sport",
      "movies"
    ],
    "attributes": {
      "AddressLine1": [
        {
          "value": "00836 W Wewlington Ave F1 5 NEW"
        }
      ],
      "AddressLine2": [
        {
          "value": "Chicago EYE NEW"
        }
      ],
      "StateProvince": [
        {
          "value": "IL NEW"
        }
      ],
      "City": [
        {
          "value": "Chicago NEW"
        }
      ],
      "Country": [
        {

```

Property	Required	Description	Type
		<pre>"value": "USA NEW" }</pre>	

		<pre> "Zip5": { "value": "60657" } } }] }, "crosswalks": [{ "type": "configuration/sources/FB", "value": "e2" }, { "type": "configuration/sources/FB", "value": "e1", "dataProvider": false }] }]</pre>		
dependentAttributes	No	Enables you to define set of attributes that depend on an entity attribute value(s). - Each JSON in the Array describes a single attribute called a <i>control attribute</i>. The control attribute URI must be specified by attributeUri parameter. All possible values of the control attribute and appropriate dependent attributes must be specified in the values Array. - Each JSON in the values Array may contain valuesList Array of strings (possible values of the control attribute) and a list of dependent attribute URIs invisibleAttributes assigned to these values. If the valuesList is not specified, then this item belongs to the undefined value of the control attribute. - If the control attribute has a value that is not specified in the values Array, then the software uses dependent attributes from the default section are used. - If the dependentAttributes section is configured for an Entity Type, then you, as a user, see the attributes assigned to current values of control attributes or any other attributes that are not mentioned in the section.	JSONArray	
lifecycleActions	No	Life Cycle Actions that must be executed for this entity lifecycle hook. Example: "lifecycleActions": { "beforeSave": ["BeforeSaveAction"] } For details, see Life Cycle Actions (LCA) Service API	JSON object	
overrideIgnorePin	No	Determines if the "pin"/"ignore" flags must be overridden after updating an attribute by the same value. The default value is true. Example: "overrideIgnorePin": "false"	Boolean	
matchBeforeCreate	No	Enables or disables on-the-fly optimization to match entities before the create operation. If optimization is enabled and a match is found by auto-rule, the software merges the new entity on-the-fly with the existing one, instead of being created separately. The default value is false. Example (enable): "matchBeforeCreate": "true"	Boolean	

Property	Required	Description	Type
forceOvForUiUpdates	No	Determines whether or not updated entity attribute values in the UI are	Boolean
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			
		so is not the operational value (OV) anymore and is not included for OV calculation. The OV in this case is determined from the remaining values based on the configured survivorship strategies. If the value is set to false, the values updated in UI are not ignored, so the configured survivorship strategies determine the OV from all available attribute values.	
useOnlyOvValuesInReferencedEntities	No	Use this option to determine whether you must use non-OV values of all attributes of the current entity type when the entity is used as a reference attribute. The default value is false; however: - If you set the value to true, then the inside reference attributes that refer to this entity use only OV values from the particular entity. In this case, if a non-OV value changes, then it is changed only in the referenced entity. So there is no need to send an event to all entities that have a relation with this one. If you search by non-OV values, it won't be found because the reference attribute won't have it. - If you enable this option, then your system performs as if the option ignoreNonOVChangesWhenUpdateThroughReferencedEntity is enabled for this entity type. This means that you cannot specify useOnlyOvValuesInReferencedEntities as true and directly specify ignoreNonOVChangesWhenUpdateThroughReferencedEntity as false. The system permits only true or no value in this case.	Boolean
ignoreNonOVChangesWhenUpdateThroughReferencedEntity	No	Use this option to determine whether you must send an event indicating that the referenced entity was changed in the case when a non-OV value was updated on a current entity. The default value is false; however: - If you set the value to true for some entity type, then during the update of an entity of another type, which uses the first entity as a reference attribute, the system will not send events indicating that all entities with references to the first entity were changed. - If you enable this option, and an OV value was changed, a message is sent, indicating that the related entity was changed. However, the message will not contain information about non-OV values. The message for the entity itself will contain all values (OV and non-OV). - If you change a Location, then ENTITY_CHANGED events for reference entities (for example, HCP/HCAs) will be sent only in the case of a change to an OV slice of the Location. This behavior is true in cases where Location is updated as a reference attribute.  Note: - If the Location is updated directly; that is, the body of a request contains the Location entity type as an object for update, then the system works as it had previously, and all related entities are notified that the Location was changed. - Reference attributes are a part of an OV slice of Location + Attributes of relationship HAS_ADDRESS.	Boolean

Property	Required	Description	Type
skipValidationForAbstractType	No	Specifies when to exclude entity types from a validation. Sometimes	Boolean

Reltio

Products

Developer resources

Roles

Applications

Objectives

during validation, you can use this parameter. When this parameter is set to **true** for an entity type, the validation rules are not applied if the entity type is abstract. This parameter takes one of the following values:

- From the highest level configuration, if it is set there.
- From the same entity type at the previous level, if it's not set at the highest level.

If the current entity type doesn't have this parameter but extends another entity type that has this parameter, the current entity type does not inherit from the parent entity type.

Example: The software takes **skipValidationForAbstractType** from the same entity type from the lower-level configuration.

In L2, the GPO entity type contains **skipValidationForAbstractType**, so there are no validation errors at this level:

```
{
  "uri": "configuration/entityTypes/GPO",
  "label": "GPO",
  "abstract": true,
  "skipValidationForAbstractType": true,
  "dataTooltipPattern": "",
  "typeColor": "#AB8800",
  "typeIcon": "images/base_type/house.png",
  "typeImage": "images/defaultImage/no-loc.png",
  "typeGraphIcon": "images/graphIcon/location-icon.png",
  "extendsTypeURI": "configuration/entityTypes/Organization",
  "attributes": [...],
  "geoLocationAttributes": [...],
  "dataLabelPattern": "{(Name)",
  "secondaryLabelPattern": "{(businessCardAttributes)",
  "survivorshipGroups": [...],
  "imageAttributeURIs": [...],
  "analyticsAttributes": [...]
}
```

In L3, the same GPO does not contain **skipValidationForAbstractType**:

```
{
  "uri": "configuration/entityTypes/GPO",
  "label": "GPO",
  "abstract": true,
  "dataTooltipPattern": "",
  "typeColor": "#AB8800",
  "typeIcon": "images/base_type/house.png",
  "typeImage": "images/defaultImage/no-loc.png",
  "typeGraphIcon": "images/graphIcon/location-icon.png",
  "extendsTypeURI": "configuration/entityTypes/Organization",
  "attributes": [...],
  "geoLocationAttributes": [...],
  "dataLabelPattern": "{(Name)",
  "secondaryLabelPattern": "{(businessCardAttributes)",
  "survivorshipGroups": [...],
  "imageAttributeURIs": [...],
  "analyticsAttributes": [...]
}
```

Result:

```
{
  "uri": "configuration/entityTypes/GPO",
  "label": "GPO",
  "dataTooltipPattern": "",
  "typeColor": "#AB8800",
  "typeIcon": "images/base_type/house.png",
  "typeImage": "images/defaultImage/no-loc.png",
  "typeGraphIcon": "images/graphIcon/location-icon.png",
  "skipValidationForAbstractType": true,
  "extendsTypeURI": "configuration/entityTypes/Organization",
  "attributes": [...],
  "geoLocationAttributes": [...],
  "dataLabelPattern": "{(Name)",
  "secondaryLabelPattern": "{(businessCardAttributes)",
  "survivorshipGroups": [...],
  "imageAttributeURIs": [...],
}
```

Property	Required	Description	Type
		"analyticsAttributes": [...]	

skipValidationForAbstractType cannot be inherited from the parent entity type.

In L2, the GPO entity type contains **skipValidationForAbstractType** and therefore, there are no validation errors at this level:

```
{
  "uri": "configuration/entityTypes/GPO",
  "label": "GPO",
  "abstract": true,
  "skipValidationForAbstractType": true,
  "dataTooltipPattern": "",
  "typeColor": "#AB8800",
  "typeIcon": "images/base_type/house.png",
  "typeImage": "images/defaultImage/no-loc.png",
  "typeGraphIcon": "images/graphIcon/location-icon.png",
  "extendsTypeURI": "configuration/entityTypes/Organization",
  "attributes": [...],
  "geoLocationAttributes": [...],
  "dataLabelPattern": "{(Name)",
  "secondaryLabelPattern": "{(businessCardAttributes)",
  "survivorshipGroups": [...],
  "imageAttributeURIs": [...],
  "analyticsAttributes": [...]
```

In L3, the GPOExtended extends**configuration/entityTypes/GPO** and does not contain **skipValidationForAbstractType**:

```
entityTypes: [
  {
    "uri": "configuration/entityTypes/GPOExtended",
    "label": "GPOExtended",
    "dataTooltipPattern": "",
    "typeColor": "#AB8800",
    "typeIcon": "images/base_type/house.png",
    "typeImage": "images/defaultImage/no-loc.png",
    "typeGraphIcon": "images/graphIcon/location-icon.png",
    "extendsTypeURI": "configuration/entityTypes/GPO",
    "abstract": true,
    "survivorshipGroups": [
      {
        "uri": "configuration/entityTypes/GPOExtended/survivorshipGroups/default",
        "default": false,
        "roles": [
          "TEST_ROLE"
        ],
        "mapping": [
          {
            "attribute": "configuration/entityTypes/GPOExtended/attributes/Address",
            "survivorshipStrategy": "Aggregation"
          },
          {
            "attribute": "configuration/entityTypes/GPOExtended/attributes/Identifiers",
            "survivorshipStrategy": "Aggregation"
          },
          {
            "attribute":
"configuration/entityTypes/GPOGPOExtended/attributes/Specialities",
            "survivorshipStrategy": "Aggregation"
          }
        ]
      }
    ]
  }
]
```

Result of Applying L3:

Error Message:

```
Attribute with URI configuration/entityTypes/GPOExtended/attributes/Address
mentioned in survivorship rules configuration is not found.
```

Example #1:

matchOvOnly: **true**, when nested attributes are at the first level, and one of the nested attributes is ignored.

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

Meta-configuration

```
"matchFieldURIs": [
  "configuration/entityTypes/HCP/attributes/L1/attributes/EmailAddress"
],
"matchOvOnly": true
```

Steps

Create entity 1

```
[
  {
    "type": "configuration/entityTypes/HCP",
    "attributes": {
      "L1": [
        {
          "value": {
            "MdmID": [
              {
                "ov": false,
                "value": "v001"
              }
            ],
            "EmailAddress": [
              {
                "ov": false,
                "value": "id@host.com"
              }
            ]
          },
          "ignored": true,
          "ov": false
        }
      ]
    },
    "crosswalks": [
      {
        "type": "configuration/sources/FB",
        "sourceTable": "HCP",
        "value": "v003"
      }
    ]
  }
]
```

The nested attribute L1 is ignored = **true**.

As a result, the nested value has OV = **false**. So, all the sub-nested-attributes of L1 (MdmID, EmailAddress) are OV = **false**.

Create entity 2

```
[
  {
    "type": "configuration/entityTypes/HCP",
    "attributes": {
      "L1": [
        {
          "value": {
            "MdmID": [
              {
                "value": "v001"
              }
            ],
            "EmailAddress": [
              {
                "value": "id@host.com"
              }
            ]
          }
        }
      ]
    },
    "crosswalks": [
      {
        "type": "configuration/sources/FB",
        "sourceTable": "HCP",
        "value": "v002"
      }
    ]
  }
]
```

```
}  
}
```

Merge entity 1 and entity 2

Result

The resulting entity after the merge has two nested attribute values. One value is nested from first entity. The other value is nested from the second entity. Due to `matchOvOnly: true`, they will not be merged even if they have the same values for `EmailAddress`.

Example #2:

`matchOvOnly: true` when the nested attributes at the first level, and both nested values are OV = `true`.

Precondition

Meta-configuration

```
"matchFieldURIs": [  
  "configuration/entityTypes/HCP/attributes/L1/attributes/EmailAddress"  
],  
"matchOvOnly": true
```

Steps

Create entity 1

```
[  
  {  
    "type": "configuration/entityTypes/HCP",  
    "attributes": {  
      "L1": [  
        {  
          "value": {  
            "MdmID": [  
              {  
                "ov": false,  
                "value": "v001"  
              }  
            ],  
            "EmailAddress": [  
              {  
                "ov": false,  
                "value": "id@host.com"  
              }  
            ]  
          },  
          "ov": false  
        }  
      ],  
      "crosswalks": [  
        {  
          "type": "configuration/sources/FB",  
          "sourceTable": "HCP",  
          "value": "v003"  
        }  
      ]  
    }  
  }  
]
```

Nested attribute L1 is OV = `true`. All sub-nested attributes of L1 (`MdmID`, `EmailAddress`) are OV = `true`.

Create entity 2

```
[  
  {  
    "type": "configuration/entityTypes/HCP",  
    "attributes": {  
      "L1": [  
        {  
          "value": {  
            "MdmID": [  
              {  
                "value": "v001"  
              }  
            ],  
            "EmailAddress": [  
              {  
                "ov": false,  
                "value": "id@host.com"  
              }  
            ]  
          },  
          "ov": false  
        }  
      ],  
      "crosswalks": [  
        {  
          "type": "configuration/sources/FB",  
          "sourceTable": "HCP",  
          "value": "v003"  
        }  
      ]  
    }  
  }  
]
```



```
        "value": "id@host.com"
      }
    ]
  }
}
```

```
      "type": "configuration/sources/FB",
      "sourceTable": "HCP",
      "value": "v002"
    }
  ]
}
```

Nested attribute L1 is OV = **true**. All sub-nested attributes of L1 (**MdmID**, **EmailAddress**) are OV = **true**.

Merge entity 1 and entity 2

Result

After the merge of the entities, two nested attributes from source entities were merged and became one nested attribute in the resulting entity.

Example #3:

The **matchOvOnly** : **true** when nested attributes are at the second level and one of the nested attributes is ignored.

Precondition

Meta-configuration

```
configuration/entityTypes/HCP/attributes/L1

"matchFieldURIs": [
  "configuration/entityTypes/HCP/attributes/L1/attributes/Email",
  "configuration/entityTypes/HCP/attributes/L1/attributes/Type"
]
"matchOvOnly": true
```

```
configuration/entityTypes/HCP/attributes/L1/attributes/L2:

"matchFieldURIs": [
  "configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Email",
  "configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Type"
]
"matchOvOnly": true
```

Steps

Create entity 1

```
{
  {
    "type": "configuration/entityTypes/HCP",
    "attributes": {
      "L1": [
        {
          "value": {
            "MdmID": [
              {
                "value": "v001"
              }
            ]
          },
          "Type": [
            {
              "value": "Office"
            }
          ],
          "Email": [
            {
              "value": "id@host.com"
            }
          ],
          "L2": [
            {
              "value": {
                "MdmID": [
                  {
                    "value": "v001_inner"
                  }
                ]
              }
            }
          ]
        }
      ]
    }
  }
}
```

```

    }
  ],
  "Type": [
    {
      "value": "Office_inner"
    }
  ]
}

```

[Reltio](#)
[Products](#)
[Developer resources](#)
[Roles](#)
[Applications](#)
[Objectives](#)

```

    "value": "id@host.com_inner"
  }
]
}
}
}
}
}
},
"crosswalks": [
  {
    "type": "configuration/sources/FB",
    "sourceTable": "HCP",
    "value": "v003"
  }
]
}
}
]

```

The nested attribute Email and L2 are OV = **true**. All sub-nested attributes of L2 are OV = **true**.

Create entity 2

```

[
  {
    "type": "configuration/entityTypes/HCP",
    "attributes": {
      "L1": [
        {
          "value": {
            "MdmID": [
              {
                "value": "v001"
              }
            ],
            "Type": [
              {
                "value": "Office"
              }
            ],
            "Email": [
              {
                "value": "id@host.com"
              }
            ],
            "L2": [
              {
                "value": {
                  "MdmID": [
                    {
                      "value": "v001_inner"
                    }
                  ],
                  "Type": [
                    {
                      "value": "Office_inner"
                    }
                  ],
                  "Email": [
                    {
                      "value": "id@host.com_inner"
                    }
                  ]
                },
                "ignored": true,
                "ov": false
              }
            ]
          }
        }
      ]
    },
    "crosswalks": [
      {
        "type": "configuration/sources/FB",
        "sourceTable": "HCP",
        "value": "v002"
      }
    ]
  }
]

```

Nested attribute L1 is OV = **true**, L2 is OV = **false**. All sub-nested attributes of L2 is OV = **false** since their upper nested are OV = **false**.

Merge entity 1 and entity 2

Meta-configuration

```
"matchFieldURIs": [
  "configuration/entityTypes/HCP/attributes/L1/attributes/Email",
  "configuration/entityTypes/HCP/attributes/L1/attributes/Type"
]
"matchOnOnly": true
```

```
"matchFieldURIs": [
  "configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Email",
  "configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Type"
]
"matchOvOnly": true
```

Steps

Create entity 1

```
}  
]
```

Create entity 2

```
[  
  {  
    "type": "configuration/entityTypes/HCP",  
    "attributes": {  
      "L1": [  
        {  
          "value": {  
            "MdmID": [  
              {  
                "value": "v001"  
              }  
            ],  
            "Type": [  
              {  
                "value": "Office"  
              }  
            ],  
            "Email": [  
              {  
                "value": "id@host.com"  
              }  
            ],  
            "L2": [  
              {  
                "value": {  
                  "MdmID": [  
                    {  
                      "value": "v001_inner"  
                    }  
                  ],  
                  "Type": [  
                    {  
                      "value": "Office_inner"  
                    }  
                  ],  
                  "Email": [  
                    {  
                      "value": "id@host.com_inner"  
                    }  
                  ]  
                }  
              }  
            ]  
          }  
        }  
      ],  
      "crosswalks": [  
        {  
          "type": "configuration/sources/FB",  
          "sourceTable": "HCP",  
          "value": "v002"  
        }  
      ]  
    }  
  }  
]
```

Nested attribute email has OV = **true**. Also, L2 has OV = **true**.

Merge entity 1 and entity 2

Result

The resulting entity after the merge will have one nested attribute **Email**. If there is one sub-nested L2 inside **Email** attribute, then the sub-nested attributes are merged.

Example #5:

The **matchOVOnly** : **true** when the nested attributes are at the second level and one of the nested at the first level is ignored.

Precondition

Meta-configuration

```
configuration/entityTypes/HCP/attributes/L1  
  
"matchFieldURIs": [  
  "configuration/entityTypes/HCP/attributes/L1/attributes/Email",  
  "configuration/entityTypes/HCP/attributes/L1/attributes/Type"
```

```
]
"matchOvOnly": true
```

```
configuration/entityTypes/HCP/attributes/L1/attributes/L2:
```

```
"matchFieldURIs": [
"configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Email",
"configuration/entityTypes/HCP/attributes/L1/attributes/L2/attributes/Type"
]
"matchOvOnly": true
```

Steps

Create entity 1

```
[
{
  "type": "configuration/entityTypes/HCP",
  "attributes": {
    "L1": [
      {
        "value": {
          "MdmID": [
            {
              "value": "v001"
            }
          ],
          "Type": [
            {
              "value": "Office"
            }
          ],
          "Email": [
            {
              "value": "id@host.com"
            }
          ],
          "L2": [
            {
              "value": {
                "MdmID": [
                  {
                    "value": "v001_inner"
                  }
                ],
                "Type": [
                  {
                    "value": "Office_inner"
                  }
                ],
                "Email": [
                  {
                    "value": "id@host.com_inner"
                  }
                ]
              }
            }
          ]
        }
      }
    ]
  }
},
{
  "type": "configuration/sources/FB",
  "sourceTable": "HCP",
  "value": "v003"
}
]
```

The nested attribute L1 and L2 both are OV = **true**. All sub-nested-attributes of L2 have OV = **true**.

Create entity 2

```
[
{
  "type": "configuration/entityTypes/HCP",
  "attributes": {
    "L1": [
      {
        "value": {
          "MdmID": [
            {
              "value": "v001"
            }
          ],
          "Type": [
            {
              "value": "Office"
            }
          ],
          "Email": [
            {
              "value": "id@host.com"
            }
          ],
          "L2": [
            {
              "value": {
                "MdmID": [
                  {
                    "value": "v001_inner"
                  }
                ],
                "Type": [
                  {
                    "value": "Office_inner"
                  }
                ],
                "Email": [
                  {
                    "value": "id@host.com_inner"
                  }
                ]
              }
            }
          ]
        }
      }
    ]
  }
},
{
  "type": "configuration/sources/FB",
  "sourceTable": "HCP",
  "value": "v003"
}
]
```

```
}
],
"Type": [
  {
    "value": "Office"
  }
]
```

```
    "value": "id@host.com"
  }
],
"L2": [
  {
    "value": {
      "MdmID": [
        {
          "value": "v001_inner"
        }
      ],
      "Type": [
        {
          "value": "Office_inner"
        }
      ],
      "Email": [
        {
          "value": "id@host.com_inner"
        }
      ]
    }
  }
],
},
"ignored": true,
"ov": false
}
]
},
"crosswalks": [
  {
    "type": "configuration/sources/FB",
    "sourceTable": "HCP",
    "value": "v002"
  }
]
}
]
```

The nested attribute L1 is OV = **false**, L2 has OV = **true**.

Merge entity 1 and entity 2

Result

The resulting entity after the merge has two nested attributes of **Email**.

Attribute ordering strategy

Attribute Ordering is a sorting strategy for an attribute and is used to visually sort multiple instances of a nested or reference attribute. Attribute ordering strategy is defined in the [Attributes configuration](#). All supported options are listed in the table below.

Strategy	Description	Attribute Configuration Examples
----------	-------------	----------------------------------

	The most recently updated attribute values will have the highest sort priority. By default, all attribute values are ordered by this strategy. The value with the maximum update date comes first. Maximum update date for the value is calculated as follows: 1. For each crosswalk bound to the value we take a singleAttributeUpdateDate. If the current crosswalk doesn't contain such a date, we take the updateDate of the whole crosswalk. Now, we have the list of update dates from each crosswalk. 2. Then, we choose the maximum date among the obtained update dates for the value. The attribute values are then sorted by such dates in descending order.	<pre>{ "uri": "configuration/entityTypes/Individual/attributes/FirstName", "name": "FirstName", "label": "First Name", "attributeOrdering": { "orderingStrategy": "LUD" }, "type": "String" },</pre>	
--	---	--	--

Strategy	Description	Attribute Configuration Examples
FieldBased	Only for	
	nested or reference. This ordering strategy defines that complex attribute values should be sorted in accordance with some attribute value inside these attributes. For example, for the nested attribute Education, we can use ordering based on the YearOfGraduation value. See the note below the table for additional information.	<pre>"name": "Education", "label": "Education", "type": "Nested", "attributeOrdering": { "orderingStrategy": "FieldBased", "fieldURI": "configuration/entityTypes/Individual/attributes/Education/attributes/YearOfGraduation" }, ...</pre>

 **Note:**

- Optionally, an order type property can be defined (for example, **orderType:DESC** or **orderType:ASC**). This indicates that the property applies an ascending or descending order to attributes values sorting. By default, ascending order is used.
- If the LUD is the same for two or more nested attributes, the sort order for those nested attributes may change each time you perform a load.

LUD Strategy Example

L3 Tenant Configuration Example: LUD Sort

```
{
  "URI": "configuration/entityTypes/Individual/attributes/FirstName",
  "name": "FirstName",
  "label": "First Name",
  "attributeOrdering": {"orderingStrategy": "LUD"},
  "type": "String"
}
```

Data Example: LUD Sort

Education					
Student	University	Degree	Year of Graduation	GPA	Last Updated Date
C	University of Toronto	MBA	2006	3.5	3/11/15
C	University of Toronto	PhD	2010	3.8	2/11/15
C	University of Minnesota	BA	2004	3.7	1/11/15

Field-based Strategy Example

L3 Tenant Configuration Example: Field-based Sort

```
    "fieldURI": "configuration/entityTypes/individual/attributes/Education",
    "name": "Education",
    "label": "Education",
    "type": "Nested",
    "attributeOrdering": {
      "orderingStrategy": "FieldBased",
      "fieldURI": "configuration/entityTypes/individual/attributes/Education/attributes/YearOfGraduation"
    },
    ...
  }
```

Data Example: Field-based Sort

Education					
Student	University	Degree	Year of Graduation	GPA	Last Updated Date
C	University of Minnesota	BA	2004	3.7	1/11/15
C	University of Toronto	MBA	2006	3.5	3/11/15
C	University of Toronto	PhD	2010	3.8	2/11/15

Attributes configuration

Attributes configuration overview

In the Reltio business model configuration, you can configure the attributes in Entity Types, Relation Types, and Interaction Types.

An attribute in the Reltio platform can hold multiple values, each associated with a contributing crosswalk. Use the [Attribute APIs](#) to manipulate individual values associated with an attribute.

Reltio supports these attribute types:

Enumerated

Simple attributes with a predefined set of values.

Lookup

Simple attributes with a predefined set of values loaded from a JSON file and thus can be easily changed.

Nested

Non-atomic attributes that can have simple or nested attributes. An example is the Education attribute that consists of atomic simple attributes such as University, Year of Graduation, and Degree.

 **Note:** If you want to merge nested attributes, [contact Support](#) to enable the `collapseNon0vNestedWhenMatch0vOnlyEnabled` parameter in your tenant configuration. By default, the flag is disabled, that is, it's set to `false`.

Reference

Non-atomic attributes that reference properties of another entity object. An example is the `Address` attribute references the `Location` entity. Multiple objects can reference the same object. Typically used to model a M:M relationship

Attribute configuration can specify all types of attributes, in any order (order is preserved). If you change an attribute type, create a new index for your tenant to obtain correct results in future searches. You can initiate a Reindex job and create a separate index using the `enableSeparateIndexing` flag. For more information, see [Reindex Data Task](#).

 **Note:** The number of attributes per entity may require specific design considerations. If you intend to create entities that have more than 500 attributes, [contact Support](#) or your Customer Success team.

If the values of the following configuration parameters are null, not defined, or are empty arrays, then these attributes inherit their values from the parent attributes:

- keyAttributeURIs

- matchFieldURIs
- doNotOverrideForSourceURIs
- immutableExceptForSources

• dependentLookupAttributes

In the following examples, the parameter values in the child configuration aren't defined, so they inherit their values from the parent configuration.

Parent Configuration

```
...entityTypes: [
  {
    "uri": "configuration/entityTypes/HCO",
    "attributes": [
      {
        "label": "Name",
        "name": "Name",
        "description": "Name",
        "type": "String",
        "hidden": false,
        "required": false,
        "attributeOrdering": {
          "orderType": "ASC",
          "orderingStrategy": "LUD"
        },
        "uri": "configuration/entityTypes/HCO/attributes/Name",
        "skipInDataAccess": false
      }
    ]
  }
]
```

Child Configuration

```
...entityTypes: [
  {
    "uri": "configuration/entityTypes/HCO",
    "attributes": [
      {
        "description": "New Simple attribute",
        "name": "A1",
        "label": "A1_label",
        "type": "String",
        "hidden": true,
        "uri": "configuration/entityTypes/HCO/attributes/A1"
      },
      {
        "label": "Changed label for Name",
        "type": "String",
        "uri": "configuration/entityTypes/HCO/attributes/Name"
      }
    ]
  }
]
```

Configuration With Inheritance

```
...entityTypes: [
  {
    "uri": "configuration/entityTypes/HCO",
    "attributes": [
      {
        "label": "A1_label",
        "name": "A1",
        "description": "New Simple attribute",
        "type": "String",
        "hidden": true,
        "attributeOrdering": {
          "orderingStrategy": "LUD"
        },
        "uri": "configuration/entityTypes/HCO/attributes/A1",
        "skipInDataAccess": false
      },
      {
        "label": "Changed label for Name",
        "name": "Name",
        "description": "Name",
        "type": "String",
        "hidden": false,
        "required": false,
        "attributeOrdering": {
          "orderType": "ASC",
          "orderingStrategy": "LUD"
        },
        "uri": "configuration/entityTypes/HCO/attributes/Name",
        "skipInDataAccess": false
      }
    ]
  }
]
```

```
    ]
  }
  ]...
```

Attributes structure

Attributes of an object are stored in the **attributes** property. It's a JSON object, where all attributes are grouped by attribute names and that has URI: **{entity URI}/attributes**. Each attribute name represents a collection of attribute instances with the structure provided in the table below. All requests to the data API must be signed with an access token. For more information, see topic [Authentication API](#).

Attributes structure

Property	Description	Type
URI	The path used to reach an attribute configuration. Format: configuration/{objectType}/attributes/{AttributeName}	String, URI
type	Type of attribute from the configuration.	String, URI
value	For simple attributes: value of attribute. Can be String, Integer, Number, or URL based on the type of attribute. For nested and reference attributes: stores subattributes.	String OR JSON object
required	Specifies whether the attribute is required (that is, during create/update, the API validates that this attribute is defined). Default value is false .	String
entityURI	Applicable only for reference attributes (see Reference attribute configuration).	Entities API URI
refEntity	Only for reference attributes. Contains " crosswalks " of referenced entity attributes and " objectURI " pointing to the referenced entity.	JSON object
refRelation	Only for reference attributes, contains " crosswalks " of referenced relation and " objectURI " pointing to the referenced relation.	JSON object
startEntityCrosswalks	Crosswalks of start entity with which this relation was created in the case when a relation was created as part of the reference attribute, or specifying start entity by crosswalk.	JSON object
endEntityCrosswalks	Crosswalks of end entity with which this relation was created in the case when a relation was created as part of reference attribute, or specifying end entity by crosswalk.	JSON object
skipInDataAccess	Controls whether the attribute is included in data access pipelines such as analytics exports and connector outputs. If true , the attribute is excluded from those exports. Default is false .	Boolean

Example:

```
{
  "URI": "entities/e1/attributes",
  "FirstName": [
    {
      "URI": "entities/e1/attributes/FirstName/1",
      "type": "configuration/entityTypes/Individual/attributes/FirstName",
      "value": "Ivan"
    },
    {
      "URI": "entities/e1/attributes/FirstName/2",
      "type": "configuration/entityTypes/Individual/attributes/FirstName",
      "value": "John"
    }
  ],
  "Education": [
    {
      "URI": "entities/e1/attributes/Education/3",
      "type": "configuration/entityTypes/Individual/attributes/Education",
      "value": {
        "University": [
```

```
{
  "URI": "entities/e1/attributes/Education/3/University/4",
  "type": "configuration/entityTypes/Individual/attributes/Education/University",
  "value":"UCSF"
},
```

[Reltio](#)[Products](#)[Developer resources](#)[Roles](#)[Applications](#)[Objectives](#)

```
    }
  ]
}
}
]
"Address":[
{
  "URI": "entities/e1/attributes/Address/6",
  "entityURI": "entities/e2",
  "type": "configuration/entityTypes/Individual/attributes/Address",
  "value":{
    "City":{
      {
        "uri": "entities/e1/attributes/Address/6/City/7",
        "type": "configuration/entityTypes/Location/attributes/City",
        "value":"New York"
      }
    }
  },
  "AddressType":[
    {
      "URI": "entities/e1/attributes/Address/6/AddressType/8",
      "type": "configuration/relationshipTypes/HasAddress/attributes/AddressType",
      "value":"Home"
    }
  ]
}
}
]
}
```

Simple and enumerated attribute configuration

The Simple and Enumerated attribute configurations have the following properties:

Property	Required	Description	Type
Reltio Products Developer resources Roles Applications Objectives			
		Format: <code>configuration/{objectType}/attributes/{AttributeName}</code>	
ID	Yes, for response JSON	System-generated id used internally to get the difference between two versions of the configurations when the configuration is updated.	String
label	Yes	Readable name of the attribute. This is the text that is shown in the UI for the attribute.	String
name	Yes	The Name of an attribute. The name that is used for the attribute internally in the system. an attribute must be unique across entities. For example, if you have entities for person, employee, and individual, you must have an attribute with the name property set to <code>firstname</code> for only one of the entities; not all three. Non-unique attribute names can cause sorting and other search-related issues.	String
hidden	No	This property is optional. It allows marking an attribute as hidden. The hidden attributes will be present in the response only when the <code>sendHidden</code> option is used in the request. This flag can be used by applications to determine when to display characteristics of attributes. For example, the Hub doesn't display attributes marked as hidden. Default: <code>false</code> .	Boolean
Description	No	Detailed description for the attribute.	String
faceted	No	If <code>true</code> , this attribute will be displayed in the +More list on the Search page when the entity is selected.	Boolean
defaultFacetedAttributes	No	If <code>true</code> , the array of attributes will be displayed in the default attributes list or as facets in the Quick Filters panel on the Search page when the entity is selected.	Array of attributes

Property	Required	Description	Type
type	Yes	Data type of the attribute, Simple, Nested, or Reference. Possible	String,
Reltio Products Developer resources Roles Applications Objectives			
		- StringNotTokenized - Long - Int - Number - Float - Boolean - Date - Timestamp - Blob - URL - Blog URL - Image URL - SSN - Id card (Passport) - SIC-code - NAICS-code - Email domains - Ticker symbol - CIK id - Stock exchange - GeoLocation	
cardinality	No	Cardinality of the attribute type. It has two sub-elements: - minValue - MaxValue During the application of a configuration, Reltio Cloud verifies that if minValue is specified, then it must be greater than or equal to zero; if maxValue is specified, then it must be greater than or equal to 1; and if both attributes are specified, then maxValue must be greater than or equal to minValue. Here's an example: <pre>{ "URI": "configuration/entityTypes/Individual/attributes/FirstName", "cardinality": { "minValue": 1, "maxValue": 2 } },</pre>  Note: The values must be Integers; that is, values such as 1.793 aren't acceptable.	Integer
values	Yes, for enumeration attributes	Array of possible values for an attribute.	Array of values (String)
defaultValue	No	This property is optional. It allows configuring default values for attributes. When you create a new profile in the UI, these attributes are added in the profile with the configured default values. You can also override the default value.	String



Note:

The **StringNotTokenized** type for a simple attribute signals that two fields must be created in the index when storing attribute values of this type in the ES index:

- normal ("name": "**San Francisco**")

- `"nameNotTokenized": "San_Francisco"` - one with post-fix `"NotTokenized"` with all value white spaces and punctuation replaced with some sign like `"_"` to avoid ES tokenization.

```
{
  "URI": "configuration/entityTypes/Individual/attributes/FirstName",
  "Name": "FirstName",
  "ID": "4",
  "label": "First Name",
  "type": "String"
}
```

Example 2:

```
{
  "URI": "configuration/entityTypes/Individual/attributes/Gender",
  "Name": "Gender",
  "ID": "5",
  "label": "Gender",
  "type": "String",
  "values": ["Male", "Female", "Unknown"]
}
```

 **Note:** Currently, `ID` parameters aren't supported by the Configuration API. Also, while configuring self-pointing relations, don't configure reference attributes as part of the relationship. For more information, see [Self Pointing Relationships](#).

Nested attribute configuration

The following table summarizes the nested attribute configuration properties:

Property	Required	Description	Type
id	Yes	Internal, system-generated ID used internally to get the difference between two versions of the configurations when the configuration is updated.	String
label	Yes	The readable name of the attribute.	String
hidden	No	This property is optional. It allows marking an attribute as hidden. The hidden attributes will be present in the response only when the sendHidden option is used in the request. This flag can be used by applications to determine when to display characteristics of attributes. For example, the Hub doesn't display attributes marked as hidden. Default: false .	Boolean
dataLabelPattern	Yes	Pattern that is used to build the attribute value label. This can include static text as well as patterns-attributes in curly brackets. Example: {FirstName} {MiddleName} {LastName}. With square brackets you can define conditional label patterns. If an attribute used inside the brackets doesn't exist, then all content inside the brackets will be skipped. Example: {FirstName} {MiddleName} {LastName}[, {Credentials}]  Note: - If dataLabelPattern isn't specified, the label is used. - If any attribute has multiple OV values, such as AddressType has values, SHIP_TO and BILL_TO, then if you assign the addressType attribute to dataLabelPattern (dataLabelPattern={addressType}), this will take only the first value SHIP_TO and not the other value(s) BILL_TO.	String
Name	Yes	Name of an attribute. Will be used in Data API (/*link*/).	String
Description	No	Detailed description for the attribute.	String
type	Yes	For Nested attribute, the value must always be Nested .	Static String or 'Nested'
cardinality	No	Cardinality of the attribute type. It has two sub-elements: minValue maxValue During the application of a configuration, Reltio Cloud verifies that if minValue is specified, then it must be greater than or equal to zero; if maxValue is specified, then it must be greater than or equal to 1; and if both attributes are specified, then maxValue must be greater than or equal to minValue. Here's an example: <pre>{ "URI": "configuration/entityTypes/Individual/attributes/FirstName", "cardinality": { "minValue": 1, "maxValue": 2 } },</pre>  Note: The values must be Integers; that is, values such as 1.793 aren't acceptable.	Integer
attributes	Yes	Array of possible nested attributes.  Caution: It's bad practice to use the same name for a nested attribute and its sub-attribute.	Array of Attribute Configuration

Property	Required	Description	Type
<code>enableNestedPartialOverride</code>	No	Set true/false to control whether nested attributes can be partially overridden. Value configured	Boolean
Reltio Products Developer resources Roles Applications Objectives			
<code>singleValue</code>	No	If true , then only one value can appear in the attribute in the entity. If you specify several different values (possibly from different sources), then all these values will be combined into one nested value (with different values for subattributes). Default: false .	Boolean
<code>singleValueByCrosswalk</code>	No	If true , then only one value for a particular source can appear in the attribute in the entity. If you specify several different values (from the same source), then all these values will be combined into one nested attribute value (with different values for subattributes). If there is partial override, such nested attribute from the request body won't substitute whole value for the dataProvider crosswalk, but it will override (partially) subattributes inside the value. Default: false .	Boolean
<code>singleValueByCrosswalkSources</code>	No	If the <code>singleValueByCrosswalk</code> property is true, you can additionally specify for which sources to use or ignore the <code>singleValueByCrosswalk</code> rule. Attributes: - include-array of Sources: specifies the exact sources for which the <code>singleValueByCrosswalk</code> rule will be applied. Other sources will be ignored. - exclude-array of Sources: specifies the exact sources for which the <code>singleValueByCrosswalk</code> rule will be ignored. For other sources it will be applied. Both parameters are optional and can be used separately or in combination.	Nested array of Sources (String)

Property	Required	Description	Type
	No		
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			

matchFieldURIsExactOrNull

matchFieldURIsIgnoreCase

They have the following format:

```
{
  "URI": "configuration/entityTypes/Individual/attributes/Education",
  "Name": "Education",
  "label": "Education",
  "type": "Nested",
  "dataLabelPattern": "{University} {Degree}, {YearOfGraduation}",
  "matchFieldURIs": [
    "configuration/entityTypes/Individual/attributes/Education/University",
    "configuration/entityTypes/Individual/attributes/Education/Degree"
  ],
  "matchFieldURIsExactOrAllNull": [
    "configuration/entityTypes/Individual/attributes/Education/YearOfGraduation",
    "configuration/entityTypes/Individual/attributes/Education/YearsInProgram"
  ],
  "matchFieldURIsExactOrNull": [
    "configuration/entityTypes/Individual/attributes/Education/GPA",
    "configuration/entityTypes/Individual/attributes/Education/Graduated"
  ],
  "matchFieldURIsIgnoreCase": true,
  "attributes": {
    ...
  }
}
```

In the example above, the following portion

```
"matchFieldURIs":
[ "configuration/entityTypes/Individual/attributes/Education/University", "configuration/entityTypes/Individual/attributes/Education/YearOfGraduation",
"matchFieldURIsExactOrAllNull": [ "configuration/entityTypes/Individual/attributes/Education/YearOfGraduation",
"configuration/entityTypes/Individual/attributes/Education/YearsInProgram"],
"matchFieldURIsExactOrNull": [ "configuration/entityTypes/Individual/attributes/Education/GPA",
"configuration/entityTypes/Individual/attributes/Education/Graduated"]
```

specifies matching "Education" with the same "University" and "Degree" subattributes, the same or both missing YearOfGraduation and YearsInProgram subattributes, and the same or any missing "GPA" and Graduated subattributes. The match rules sections are connected by AND. However, each section is optional and can be used in any combination with other sections or separately.

 **Note:** matchFieldURIs, matchFieldURIsExactOrAllNull, matchFieldURIsExactOrNull rules are connected by AND. Each section is optional and can be used in any combination with the others or separately. If the match rule section isn't defined, then nested attributes will merge if all of their subattributes have the same value.

If the matchFieldURIs section is defined (as described above for the Education nested attribute), then the following apply:

- Merge if all matchFields are the same, but there are some other different subattributes.
- Do not merge attributes having all the same fields if not all of matchFields are defined. The reason is there isn't enough information to consider these attributes equal.

If the "matchFieldURIsExactOrAllNull" section is defined, (as described above for the "Education" nested attribute), then the following apply:

- Merge if all matchFields are the same or are missing in both nested attributes, but there are some other different subattributes.
- Don't merge attributes having all the same fields if some of matchFields are defined only in one attribute.

If the matchFieldURIsExactOrNull section is defined (as described above for the Education nested attribute), then the following applies:

- Merge if all matchFields are the same or missing in any nested attribute, but there are some other different subattributes.

By default, the matchFieldURIs nested attribute property is case sensitivity for attribute values, when performing matching for matchField configuration. Set matchFieldURIsIgnoreCase to true for matchFieldURIs, matchFieldURIsExactOrAllNull, and matchFieldURIsExactOrNull configuration to get case insensitive matching.

Property	Required	Description	Type
		Example 1:	
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			
		Edu1 { "University"- "ITMO", "Degree"- "BA of Technology", "SchoolName"- "School1" } must merge with this inserted attribute: Edu2 { "University"- "ITMO", "Degree"- "BA of Technology", "SchoolName"- "School2" } in one "Education" attribute in the entity because: - all matchFieldURIs fields (University, Degree) are the same (YearOfGraduation, YearsInProgram, Graduated, and GPA) are missing in both attributes, even if there's a different attribute "SchoolName". Example 2: Don't merge attributes having all the same fields if not all of the fields from matchFieldURIs are defined. Edu1 { "University"- "ITMO", "GPA"- "5" } must not merge with the inserted attribute Edu2 { "University"- "ITMO", "GPA"- "5" } in one Education attribute in the entity because there's no field ("Degree") defined in these attributes. Example 3: Don't merge attributes having all the same fields if some fields from matchFieldURIsExactOrAllNull are defined in only one attribute. Edu1 { "University"- "ITMO", "Degree"- "BA of Technology", "YearOfGraduation"- "5", "YearsInProgram"- "4", "GPA"- "4.9", "Graduated"- "true" } must not merge with the inserted attribute. Edu2 { "University"- "ITMO", "Degree"- "BA of Technology", "YearsInProgram"- "4", "GPA"- "4.9", "Graduated"- "true" } in one "Education" attribute because there's no field ("YearOfGraduation") defined in the Edu2 attribute, although all other attributes are the same. Example 4: Merge attributes if all fields from matchFieldURIs are the same, all fields from matchFieldURIsExactOrAllNull are missing, and some of the fields from matchFieldURIsExactOrNull are defined only in one attribute. Edu1 { "University"- "ITMO", "Degree"- "BA of Technology", "GPA"- "4.9", "Graduated"- "true" } must merge with the inserted attribute.	

Property	Required	Description	Type
ReltioProductsDeveloper resourcesRolesApplicationsObjectives			
		in one "Education" attribute because: - all matchFieldURIs fields {University, Degree} are the same, {YearOfGraduation and YearsInProgram} are missing in both attributes, and {GPA} is the same in both attributes, although {Graduated} is missing in the Edu2 attribute.  Note: Nested attributes merge when a new attribute is inserted or a nested attribute is updated.	
match0vOnly	No	Optional property that specifies whether non-OV values of subattributes would be considered when merging different values of the nested attribute. Default value is false. If the value is set to true, that is, if match0vOnly is enabled, then match rules are applied only to sub-attribute values with OV=true, and sub-nested attributes that only have end-dated crosswalks are considered to be different from attributes that don't have values or have values with at least one active crosswalk. If the value is false, that is, if match0vOnly is disabled, then all values of the subattribute are considered. For example, the following value Edu1 { "University": "ITMO", "Degree": ["BA of Technology" (ov:true), "Bachelor of Technology" (ov: false)] } must merge with the inserted value Edu2 { "University": "ITMO", "Degree": ["BA of Technology" (ov:true), "BA of IT" (ov: false)] } if "match0vOnly" is true; otherwise, it mustn't be merged.  Note: This property can't be used with 'flat0vCalculation' enabled, and neither with 'OtherAttributeWinnerCrosswalk' survivorship strategy with outer 'primaryAttributeUri' used for subattributes of a nested .	

Example:

```
{
  "URI": "configuration/entityTypes/Individual/attributes/Education",
  "Name": "Education",
  "type": "Nested",
  "dataLabelPattern": "{University} {Degree}, {YearOfGraduation}",
  "singleValueByCrosswalk": true,
  "singleValueByCrosswalkSources": {
    "exclude": [
      "configuration/sources/Reltio"
    ]
  },
  "matchFieldURIs": [
    "configuration/entityTypes/Individual/attributes/Education/University",
    "configuration/entityTypes/Individual/attributes/Education/Degree"
  ],
  "match0vOnly": true,
  "attributes": {
    {
      "URI": "configuration/entityTypes/Individual/attributes/Education/University",
      "ID": "1",
      "Name": "University",
      "label": "University",
      "type": "String"
    },
    {
      "URI": "configuration/entityTypes/Individual/attributes/Education/Degree",
      "ID": "2",
      "Name": "Degree",
      "label": "Degree",
      "type": "String"
    },
    {
      "URI": "configuration/entityTypes/Individual/attributes/Education/YearOfGraduation",
      "ID": "3",
      "Name": "YearOfGraduation",
```

```
"label": "Year of graduation",
"type": "Int"
},
{
  "URI": "configuration/entityTypes/Individual/attributes/Education/GPA",
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

```
}
  ]
}
```

 **Note:** Currently, **ID** parameters aren't supported by the Configuration API.

Reference attribute configuration

The Reference attribute configuration has the following properties:

 **Note:** You can't configure one entity type attribute as a dependent attribute of another entity type. For example, it isn't possible to configure an HCO's attribute as a dependent attribute of HCP.

Property	Required	Description	Type
uri	Yes	URI of the attribute. It must be a valid URI. Example: <code>configuration/{objectType}/attributes/{AttributeName}</code> .	String, URI
Reltio Products Developer resources Roles Applications Objectives			
		<code>configuration/{objectType}/attributes/{AttributeName}</code>	
ID	Yes, for response JSON	System-generated id used internally to get the difference between two versions of the configurations when the configuration is updated.	String
label	Yes	Readable name of the attribute.	String
Name	Yes	Name of the attribute. Used in the Data API (<code>/*link*</code>).	String
Description	No	Detailed description for the attribute.	String
hidden	No	This property is optional. It allows marking an attribute as hidden. The hidden attributes will be present in the response only when the <code>sendHidden</code> option is used in the request. This flag can be used by applications to determine display characteristics of attributes. For example, the Hub doesn't display attributes marked as hidden. Default: <code>false</code> .	Boolean
type	Yes	For reference attributes the value must always be <code>reference</code> .	Static String "Reference"
cardinality	No	Cardinality of the attribute type. It has two sub-elements: <code>minValue</code> <code>maxValue</code> During the application of a configuration, Reltio Cloud verifies that if <code>minValue</code> is specified, then it must be greater than or equal to zero; if <code>maxValue</code> is specified, then it must be greater than or equal to 1; and if both attributes are specified, then <code>maxValue</code> must be greater than or equal to <code>minValue</code>. Here's an example: <pre>{ "URI": "configuration/entityTypes/Individual/attributes/FirstName", "cardinality": { "minValue": 1, "maxValue": 2 } };</pre>  Note: The values must be Integers; that is, values such as 1.793 aren't acceptable.	Integer
relationshipTypeURI	Yes	The URI of a relationship type used in the reference attribute.	String, Relationship Type URI

Property	Required	Description	Type
relationshipLabelPattern	Yes	The pattern that is used to build the attribute relationship value	String, Pattern
Reltio Products Developer resources Roles Applications Objectives			
		Example: <pre>{FirstName} {MiddleName} {LastName}</pre> With square brackets you can define conditional label patterns: if an attribute used inside the brackets doesn't exist, then all content inside the brackets will be skipped. Example: <pre>{FirstName} {MiddleName} {LastName}[, {Credentials}]</pre>  Note: If <code>relationshipLabelPattern</code> isn't specified, label is used.	
referencedEntityTypeURI	Yes	The URI of entity type or role that is referenced from an entity though <code>relationshipTypeURI</code> .	String, Entity Type, or Role URI
referencedAttributeURIs	Yes	A Reference attribute doesn't have its own attributes. It's using attributes of a relationship and referenced entity. In this property, an array of relationship and referenced entity URIs is the set that will be used as part of the reference attribute.	Array of URIs for relationship and referenced entity attributes
doNotOverrideForSourceURIs	No	If there is profile override, if the reference attribute isn't provided in a request, then the existing reference attributes from sources (specified in this parameter) won't be deleted. You'll need to delete relationships that are no longer needed.	Array of URIs for Sources
referenceAttributeDirection	No	Specifies whether the reference attribute must appear in the entity when a relationship of type <code>relationshipTypeURI</code> is present. Possible values: both: (default) The reference attribute appears if a relationship of type <code>relationshipTypeURI</code> is present. parentToChild: The reference attribute appears if a relationship of type <code>relationshipTypeURI</code> is present and if the entity is a startEntity in the relation. childToParent: The reference attribute appears if a relationship of type <code>relationshipTypeURI</code> is present and if the entity is an endEntity in the relation. This attribute can be useful when the entity and the referenced entity are of the same type. For example, there's a reference attribute built on an "OwnedOrganizations" relation, which connects organizations "Org1" and "Org2". In this case, if there are two organizations and a relation between them (where, for example, "Org1" is startEntity, and "Org2" is endEntity of the relation), then on "Org1", in the "Legal Entities Hierarchy" facet, you'll see a tree, where "Org1" is a parent of "Org2". Also, "Org1" will have a reference attribute equal to "Org2". At the same time, on "Org2" you'll see that "Org2" has a reference attribute equal to "Org1". If <code>referenceAttributeDirection=parentToChild</code>, then "Org1" will have a reference attribute and "Org2" won't have it.	Static String: "both" , "parentToChild" , or "childToParent"
immutable	No	Defines the Immutable Reference Attributes . This parameter overrides the <code>immutableRefAttrsByDefault</code> parameter from the tenant configuration for this particular attribute.	Boolean
immutableForSources	No	Used to define an array of source systems (URIs and IDs) for which it isn't possible to change the value of subattributes that come from the referenced entity. However, you can change the values from the relation.	Array of URIs for Sources (String)

Property	Required	Description	Type
<code>immutableExceptForSources</code>	No	Defines a list of sources that will ignore the <code>immutable</code>	Array of URIs for

Reltio
Products
Developer resources
Roles
Applications
Objectives

 **Note:** The three properties, `immutable`, `immutableForSources`, and `immutableExceptForSources` can also be configured at the tenant level globally. To do this, contact Support.

Example:

```
{
  "URI":"configuration/entityTypes/Individual/attributes/Address",
  "Name":"Address",
  "label":"Address",
  "ID":"9",
  "type":"Reference",
  "referencedEntityTypeURI":"configuration/entityTypes/Location",
  "relationshipTypeURI":"configuration/relationshipTypes/HasAddress",
  "relationshipLabelPattern":{"configuration/relationshipTypes/HasAddress/attributes/AddressType"},
  "doNotOverrideForSourceURIs" : [
    "configuration/sources/DEA",
    "configuration/sources/Veeva"
  ],
  "referencedAttributeURIs":[
    "configuration/relationshipTypes/HasAddress/attributes/AddressType",
    "configuration/entityTypes/Location/attributes/City"
  ]
}
```

 **Note:** Currently, `ID` parameters aren’t supported by the Configuration API.

 **Note:** Reference attributes are automatically included during bulk update and attribute update tasks if your tenant has the `alwaysLoadReferenceAttributesInBulkUpdateTask` parameter set to `true` in the `optionalParameters` section of the physical configuration. When the parameter is not explicitly set, the system treats it as enabled by default. To disable this behavior, you must explicitly set the parameter to `false`. Because this is a physical configuration parameter, you must contact Reltio Support and submit a Zendesk ticket to request any updates. See topic, [Need some help?](#) .

Attribute ordering and attribute survivorship

Attribute values ordering and attribute values survivorship are both complex sorting strategies, but have different usage and configuration. The survivorship results of both these strategies are transient and don’t interfere with each other. Attribute survivorship strategy results (OV values) are used by the UI to customize which attribute values to display. Attribute values order in output is determined by attribute ordering strategy.

Attribute ordering strategy is defined in the attribute configuration. All supported options are given in the table below.

Strategy Name	Description	Attribute Configuration Example
ReltioProductsDeveloper resourcesRolesApplicationsObjectives		
	Last Update Date strategy. The most recently updated attribute values have the highest sort priority. By default, all attribute values are ordered by this strategy, and the value with the maximum update date is first in the listing of ordered values. The maximum update date for the value is calculated as follows: - For each crosswalk bound to the value, try to take a "singleAttributeUpdateDate". However, if the current crosswalk doesn't contain such a date, then take the "updateDate" of the entire crosswalk so that you obtain the list of update dates from each crosswalk. - Then, choose the maximum date among the obtained update dates for the value. After the update is calculated, attribute values are sorted by such dates in descending order.	<pre>{ "URI":"configuration/entityTypes/Individual/attributes/FirstName", "Name":"FirstName", "label":"First Name", "attributeOrdering":{" "orderingStrategy":"LUD" }, "type":"String" };</pre>
FieldBased	Only for complex attributes such as nested or reference. This type of ordering strategy defines that complex attribute values must be sorted in accordance with some attribute value inside this attribute. For example, for nested attribute Education, you can use ordering based on the YearOfGraduation value. Note: Optionally, you can define the order type property as "orderType" : "DESC" or "orderType": "ASC". This property applies ascending or descending order to attributes values sorting. By default, the ascending order is used.	<pre>{ "URI":"configuration/entityTypes/Individual/attributes/Education", "Name":"Education", "label":"Education", "type":"Nested", "attributeOrdering":{" "orderingStrategy":"FieldBased", "fieldURI":"configuration/entityTypes/Individual/attributes/Education/attributes/YearOfGraduation" }, ... }</pre>

Attribute Survivorship Rules are rules that drive the calculation of the operation values (OV) for an attribute among the values that have been contributed by various sources to the attribute. Attribute Ordering is a sorting strategy for an attribute and is used to visually sort multiple instances of a nested or reference attribute. For more information, see [Attribute ordering strategy](#) and [Design survivorship rules](#).

Configuring Fallback Strategies for Survivorship Rules

You can set a fallback strategy for survivorship rules. For example, if you have a source system survivorship rule for an attribute, and you have two records from the same source system involved in a

merge, Reltio would generate two Operational Values (OVs) for the attribute. To get a single OV, you can specify the fallback strategy as a tiebreaker .

Fallback strategies can be any attribute survivorship rule (see [Design survivorship rules](#) for more

its own set of Fallback Strategies defined separately.

deprecated, and you now use **fallbackStrategies**.

To configure fallback strategies, you can use the **survivorshipGroups** section of the L3 configuration file to specify the values for **fallBackStrategies**.

Example

Fallback strategies provide flexibility to change an OV rule for some other attribute. In the following example, in case of fallback for **LastName**, you can change the rule for the **FirstName** attribute.

```
{
  "survivorshipGroups":[
    {
      "URI":"configuration/entityTypes/HCP/survivorshipGroups/default",
      "default":true,
      "mapping":[
        {
          "attribute":"configuration/entityTypes/HCP/attributes/FirstName",
          "sourcesUriOrder":[
            "configuration/sources/HMS",
            "configuration/sources/AHA",
            "configuration/sources/Reltio"
          ],
          "survivorshipStrategy":"SRC_SYS"
        },
        {
          "attribute":"configuration/entityTypes/HCP/attributes/MiddleName",
          "survivorshipStrategy":"OtherAttributeWinnerCrosswalk",
          "primaryAttributeUri":"configuration/entityTypes/HCP/attributes/FirstName"
        },
        {
          "attribute":"configuration/entityTypes/HCP/attributes/LastName",
          "survivorshipStrategy":"OtherAttributeWinnerCrosswalk",
          "primaryAttributeUri":"configuration/entityTypes/HCP/attributes/FirstName",
          "fallbackStrategies":[
            {
              "attribute":"configuration/entityTypes/HCP/attributes/LastName",
              "survivorshipStrategy":"SRC_SYS",
              "sourcesUriOrder":[
                "configuration/sources/HMS",
                "configuration/sources/Reltio",
                "configuration/sources/AHA"
              ]
            }
          ]
        }
      ],
      "fallbackUsingCriteria":"ZERO_OR_MORE_THAN_ONE"
    }
  ]
}
```

You can specify a **sourcesForOV** list specifically for a fallback strategy. This list defines the source systems that are used to compute the Operational Value when fallback is triggered. The fallback **sourcesForOV** list can be different from the primary strategy's source list. If **sourcesForOV** is not defined for the fallback, the system defaults to evaluating values from the full fallback context.

Note: The default value of the new survivorship strategy option **fallbackUsingCriteria** is **MORE_THAN_ONE**, which is used for backward compatibility. **ZERO_OR_MORE_THAN_ONE** means that the fallback rule applies to the result of the primary survivorship rule if there are either zero or more than one Operational Values.

Configuring Survivorship Rules in the L3

You can define the survivorship rules in the L3 configuration file as well as in the Reltio UI. And any modifications made by a user in the Reltio UI will affect the definitions in the L3 accordingly. Below is a screenshot of an example section of the L3 configuration file for Survivorship.

Updates must be POSTed to the specific entityType that is being updated.

Reltio Products **Developer resources** Roles Applications Objectives

```

8  "sourcesUriOrder": [
9    "configuration/sources/PASS",
10   "configuration/sources/IREP",
11   "configuration/sources/MCOS"
12 ]
13 }
14 "attribute": "configuration/entityTypes/ProfessionalAssociation/attributes/datefounded",
15 "survivorshipStrategy": "LUD"
16 }
17 "attribute": "configuration/entityTypes/ProfessionalAssociation/attributes/noMembers",
18 "survivorshipStrategy": "LUD"
19 }
20 "attribute": "configuration/entityTypes/ProfessionalAssociation/attributes/publications",
21 "survivorshipStrategy": "Aggregation"
22 }
23 }
24 }
25 }
26 }

```

Specify the survivorship strategy for each attribute e.g.*:

- SRC_SYS: Source
- LUD (Last Update date): Recency
- Frequency
- Aggregation

If the strategy is SRC_SYS, specify the priority of the sources.

Here are examples of attribute survivorship strategy configuration in the L3 configuration file.

Attribute Survivorship Strategy Configuration Example

```
{
  "URI":"configuration/entityTypes/Individual/attributes/MiddleName",
  "name":"MiddleName",
  "label":"Middle Name",
  "type":"String",
  "survivorshipStrategy":"SRC_SYS"
}
```

Complex Attribute Survivorship Strategy Configuration Example

```
{
  "URI": "configuration/survivorshipStrategies/TwitterSRCSysStrategy",
  "parentStrategyURI": "configuration/survivorshipStrategies/SRC_SYS",
  "label": "Twitter Custom SRC_SYS Strategy",
  "mainSourceType": "configuration/sources/TWITTER"
}
```

Record Values-Based Source Priority Strategy Example

```
{
  "URI": "configuration/survivorshipStrategies/CleanserWinsStrategy",
  "label": "Record values based source priority OV rule (based on ReltioCleanser)",
  "winnerSourceType": "configuration/sources/ReltioCleanser"
}
```

Other Attribute Winner Crosswalk Strategy Example

```
{
  "attribute": "configuration/entityTypes/HCP/attributes/LastName",
  "survivorshipStrategy": "OtherAttributeWinnerCrosswalk",
  "primaryAttributeUri": "configuration/entityTypes/HCP/attributes/FirstName"
}
```

Fallback Strategy with sourcesForOV Example

This example shows how to configure a fallback survivorship strategy using `sourcesForOV` to control which sources are evaluated during fallback. The fallback `sourcesForOV` list can be different from the primary strategy's source list.

```
{
  "attribute": "configuration/entityTypes/ProfessionalAssociation/attributes/Name",
  "survivorshipStrategy": "SRC_SYS",
  "sourcesForOV": [
    "configuration/sources/PASS",
    "configuration/sources/REP",
    "configuration/sources/MODS"
  ],
  "fallbackStrategies": [
    {

```

```
"attribute": "configuration/entityTypes/ProfessionalAssociation/attributes/Name",
"survivorshipStrategy": "SRC_SYS",
"sourcesForOV": [
  "configuration/sources/HR",
  "configuration/sources/CustomCRM"
```



Crosswalk Dates API

Use the crosswalk dates APIs to get publish date or set publish date for a particular source system.

The following APIs are available as part of crosswalk dates APIs:

- [Get Source Publish Date for Source System](#)
- [Set Source Publish Date for Source System](#)

Delete a Source System

You can delete the source system by posting a request. The response returns a JSON that displays the attributes of the removed source type and the result of the deletion request.

Request

```
DELETE {TenantURL}/{source system URI}
```

Parameters

Type	Name	Required	Description
Header	Content-Type	Yes	The value should be "Content-Type: application/json"
Header	Authorization	Yes	The value should be "Content-Type: application/json"
Tasks	option	No	Valid values include purgeAttributes (default) Deletes all attribute values for a source, but preserves the crosswalks and the source used for the configuration. purgeAllData Deletes all attribute values and crosswalks for a source, and the source used for the configuration.

 **Note:** To avoid the validation exception, if the source that you want to delete exists in any other section apart from the *sources* section, you must manually delete the source before posting the [Delete Source](#) ^{API} API request. For example, the **sourcesUriOrder** or **immutableForSources** section.

Response

If the source type is deleted successfully, then the response contains a JSON with the details of the deleted source type. The JSON will also contain a **Response** field, which displays a comment about the delete operation and the response definition of the background task that has started.

 **Note:** The response contains the definition of the background task only if the **purgeAllData** task is set to true.

The task status can be retrieved using the [Get Tasks History for Tenant](#) ^{API} API. For more information, see [Get Task by ID for Tenant](#).

Example Request

DELETE {TenantURL}/configuration/sources/AHA?option=purgeAttributes

```
{
  "sourceType":{
    "URI":"configuration/sources/AHA",
    "label":"AHA",
    "description":"AMERICAN HOSPITAL association",
    "abbreviation":"AHA",
    "icon":"images/source/source_s.png"
  },
  "Response":"Source type 'AHA' deletion background task scheduled. To see its
status please use the tasks endpoints.",
  "Background task":{
    "numberOfProcessedObjects":0,
    "name":"Purge data by 'AHA' source type background task. Started by mike",
    "tenant":"tenant",
    "started":"05-27-2014 13.55",
    "throughput":0,
    "current":"05-27-2014 13.55",
    "purgeDataOnly"="true"
  }
}
```

 **Note:** In the request, if you haven't included any options, or have included `option=purgeAttributes`, then the response displays the `purgeDataOnly` as `true` to denote only values are deleted. If you included the `option=purgeAllData`, then the response displays the `purgeDataOnly` as `false` to denote that data and crosswalks are deleted.

GET Configuration

This operation returns the configuration for a tenant, with inheritance from the configuration layers. For more information on configuration inheritance, see [Tenant configuration inheritance across layers](#). To retrieve specific object type, see [Manage configuration by object type](#).

Request

GET {TenantURL}/configuration

See this operation  on the Developer Portal.

Parameters

Type	Name	Required	Description
Path	<code>TenantURL</code>	Yes	Defines tenant that should be used to return configuration
Query	<code>select</code>	No	Identifies what elements of configuration should be returned. It is possible to return: <code>justentityTypes</code> , <code>groupTypes</code> , <code>relationTypes</code> , etc.

Response

If there is a configuration defined for a tenant specified in Header of a request, then a JSON Object representing this configuration is returned. If there is no configuration, an Error is returned.

309

Tenant '{0}' not defined in a system

310

Failed to process Reltio Business Model JSON loaded from Reltio data storage

Example Request

GET {TenantURL}/configuration

Example Response

GET {TenantURL}/configuration

Body:{

```
"entityTypes":[
  {
    "URI":"configuration/entityTypes/Individual",
    "label":"Individual",
    "name":"Individual",
    "id":"1",
    "description":"These entity types for representing people",
    "attributes":[
      {
        "URI":"configuration/entityTypes/Individual/attributes/FirstName",
        "name":"FirstName",
        "label":"First Name",
        "id":"2",
        "type":"String"
      }
    ]
  }
]
```

Get Configuration (No Inheritance)

This operation returns a configuration for a tenant without inheritance with verticals.

Request

GET {TenantURL}/configuration/_noInheritance

Parameters

Type	Name	Required	Operation specific	Description
Path	TenantURL	Yes	No	Defines the tenant that should be used to return the configuration.

Response

If there is a configuration defined for a tenant specified in a request, a JSON Object representing this configuration (without inheritance with verticals) is returned. If there is no configuration, an error is returned.

Example Request

GET {TenantURL}/configuration/_noInheritance

Example Response

```
{
  "label":"Test Business Model",
  "description":"Test Business Model Configuration-used to describe Configuration API
format",
  "schemaVersion":"1",
  "entityTypes":[
    {
      "URI":"configuration/entityTypes/Individual",
      "label":"Individual",
      "name":"Individual",
      "id":"1",
      "description":"These entity types for representing people",
      "attributes":[
        {
          "URI":"configuration/entityTypes/Individual/attributes/FirstName",
          "name":"FirstName",
          "label":"First Name",
          "id":"2",
          "type":"String"
        }
      ]
    }
  ]
}
```

```
    ]
  }
}
```

This operation returns history snapshots of configurations with some additional information: timestamp, when configuration was changed, and who made this change.

 **Note:** The configuration snapshots in the response are configurations with inheritance with verticals

Request

```
GET {TenantURL}/configuration/_history
```

Parameters

Name		Required	Operation specific	Description
Path Variable	Tenant	Yes	No	Defines the tenant used to return configuration
Query	max	No		Positive Integer value to identify the maximum number of configuration history snapshots to return in a response. Can be used to organize pagination in combination with the "offset" parameter. Default value is 10.
	offset	No		Positive Integer value to identify starting what element in a result set should be returned in a response. Can be used to organize pagination in combination with the "max" parameter. Default value is 0.

Response

A JSON Array of objects representing the configurations snapshots with additional information.

Example Request

```
GET {TenantURL}/configuration/_history?offset=6&max=3
```

Example Response

```
[
  {
    "configuration":{
      ...
    }
    "timestamp":1412798650665,
    "updatedBy":"user1"
  },
  {
    "configuration":{
      ...
    }
    "timestamp":1412798631159,
    "updatedBy":"user2"
  },
  {
    "configuration":{
      ...
    }
    "timestamp":1412798601708,
    "updatedBy":"user3"
  }
]
```

GET/

/sourcePublishDate

{ "sourcePublishDate" : 1378187168000 }

Response

Interaction Types

The Interaction Types section of [Configuration API](#) is responsible for defining types of interactions that are recognized by a tenant configuration. An interaction is generally any kind of transaction-like event that has a date and time associated with it. These could be purchases, prescriptions, claims, call center activity, etc.

Object Structure

A collection of Interaction Types can be defined for a tenant configuration. Each Interaction Type has the following properties:

Properties

Property	Description	Type
URI	Path that is used to reach the interaction type. Format: <code>configuration/interactionTypes/{TypeName}</code> Required.	String, URI
id	Internal, system-generated ID. Used internally to get the difference between two versions of configurations when a configuration is updated. Required for the response JSON.	String
label	Readable name of the Interaction Type. Required.	String
description	Description for the Interaction type.	String
extendsTypeURI	URI of an Interaction Type which is extended by this type.	String, URI
memberTypes	A collection of possible member types. Examples can be: Participant, Organizer. Member type properties: • URI: String • name: String • id: String • label: String • objectTypeURI: Array of URIs of entity types and roles that can be members of the type. • minOccurs: Integer, equal or greater than 0. Identifies how many members can be of this member type. Default is 0.	Complex
attributes	Attributes that are defined as standard for this type. Attributes can be of simple, nested and reference type. For details, refer to Attributes configuration .  Note: Not every interaction object will have all attributes defined in its interaction type. Also, an interaction object can have some attributes that are not defined in the configuration-custom attributes.	JSON object of Attributes Configuration
ignoreUniqueness	If true , Data Access API and compaction will not perform uniqueness resolution (de-duplication) for interactions of this type. If false , regular uniqueness resolution is performed. Default is false .	Boolean

Interaction Type: Meeting

Example of a **Meeting** Interaction Type configuration:

Reltio Products **Developer resources** Roles Applications Objectives

```

    "id": "31",
    "label": "Meeting",
    "attributes": {
      "URI": "configuration/interactionTypes/Meeting/attributes/Place",
      "name": "Place",
      "id": "32",
      "label": "Place",
      "type": "String"
    },
    {
      "URI": "configuration/interactionTypes/Meeting/attributes/Notes",
      "name": "Notes",
      "id": "33",
      "label": "Notes",
      "type": "String"
    }
  ],
  "memberTypes": [
    {
      "URI": "configuration/interactionTypes/Meeting/memberTypes/participants",
      "name": "participants",
      "id": "34",
      "label": "participants",
      "objectTypeURI": ["configuration/entityTypes/Individual"],
      "minOccurs": 1
    },
    {
      "URI": "configuration/interactionTypes/Meeting/organizers",
      "name": "organizers",
      "label": "organizers",
      "id": "35",
      "objectTypeURI": ["configuration/entityTypes/Individual"],
      "label": "organizers",
      "minOccurs": 1
    }
  ]
}

```

 Note: Currently, "id" parameters are not supported by the [Configuration API](#).

Interaction Type: Prescription

Example of a **Prescription** Interaction Type configuration:

```

"interactionTypes": {
  "URI": "configuration/interactionTypes/Prescription",
  "label": "Prescription",
  "ignoreUniqueness": true,
  "memberTypes": {
    "URI": "configuration/interactionTypes/Prescription/memberTypes/Beneficiary",
    "label": "Beneficiary",
    "name": "Beneficiary",
    "objectTypeURI": "configuration/entityTypes/Beneficiary"
  }, {
    "URI": "configuration/interactionTypes/Prescription/memberTypes/Product",
    "label": "Product",
    "name": "Product",
    "objectTypeURI": "configuration/entityTypes/Product",
    "minOccurs": 2
  }
}
}

```

Manage configuration by object type

Use the [Configuration API](#) to manage configuration objects by object type. These APIs support list retrieval for supported configuration object types, retrieval of individual object definitions, and creation, update, or deletion of specific object definitions.

Supported operations

The following operations are available for configuration objects by object type:

- Get an object definition
- Get all objects of a specific object type
- Add an object definition
- Update an object definition
- Delete an object definition

Access control rules

--	--

Get all objects of a specific object type

HTTP method and endpoint

```
GET {TenantURL}/configuration/{objectType}?inheritance=true&false
```

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
objectType	Yes	The configuration object type to retrieve. Supported values are <code>entityTypes</code>, <code>attributeTypes</code>, <code>hierarchyTypes</code>, <code>relationTypes</code>, <code>groupTypes</code>, <code>graphTypes</code>, <code>roles</code>, <code>interactionTypes</code>, <code>sources</code>, <code>groupingTypes</code>, <code>categoryTypes</code>, and <code>changeRequestTypes</code>.  Note: Only one <code>changeRequestType</code> can be configured at a time.

Include the following header in the request.

Query parameters

Name	Required	Description
inheritance	No	Determines whether inherited fields from base types are included in the response. Accepted values are <code>true</code> and <code>false</code> . Default: <code>true</code> .

Example request

The following example retrieves all **entityTypes** and includes inherited properties.

```
GET {TenantURL}/configuration/entityTypes?inheritance=true Authorization: <access_token>
```

Example response

The exact response payload depends on the requested object type. The following examples show the response pattern for successful and error outcomes.

Successful response

```
HTTP/1.1 200 OK Content-Type: application/json <configuration definition for the requested object type>
```

Error response

```
HTTP/1.1 400 Bad Request Content-Type: application/json <TYPE_IS_NOT_FOUND>
```

Get an object definition

Use the [Configuration API](#) to retrieve the full configuration definition for a specific type within a supported object type. By default, the response includes inherited properties from base types. To exclude inherited properties, set the **inheritance** query parameter to **false**.

HTTP method and endpoint

Use the following HTTP method and endpoint path to submit the request:

```
GET {TenantURL}/configuration/{objectType}/{typeName}?inheritance=true|false
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL for your tenant.
objectType	Yes, for the general object type endpoint	The configuration object type that contains the definition to retrieve. Supported values for this endpoint are entityTypes, groupTypes, relationTypes, graphTypes, interactionTypes, changeRequestTypes, and categoryTypes. This endpoint does not support sources.  Note: Only one changeRequestType can be configured at a time.
typeName	Yes	The name of the specific type definition to retrieve.

Request headers

Include the following header in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No	Determines whether inherited fields from base types are included in

Example request

The following example retrieves the configuration definition for the **Individual** entity type and includes inherited properties.

```
GET {TenantURL}/configuration/entityTypes/Individual?inheritance=true
Authorization: <access_token>
```

The following example retrieves the configuration definition for the **ReviewRequest** change request type.

```
GET {TenantURL}/configuration/changeRequestTypes/ReviewRequest?inheritance=true
Authorization: <access_token>
```

Example response

The exact response payload depends on the requested object type and type name. The following examples show the response pattern for successful and error outcomes.

Successful response

```
HTTP/1.1 200 OK
Content-Type: application/json

<configuration definition for the requested type>
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND>
```

Add an object definition

Use the [Configuration API](#) to create a new configuration definition for a supported object type. This operation requires a JSON payload that contains the new definition.

HTTP method and endpoint

Use the following HTTP method and endpoint paths to submit the request:

```
POST {TenantURL}/configuration/{objectType}/{typeName}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
ReltioProductsDeveloper resourcesRolesApplicationsObjectives		
	general object type endpoint	endpoint are entityTypes , groupTypes , relationTypes , graphTypes , interactionTypes , changeRequestTypes , and categoryTypes .  Note: Only one changeRequestType can be configured at a time.
typeName	Yes	The name of the type definition to create.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes	application/json

Example request

The following example creates an **Individual** entity type definition.

```
POST {TenantURL}/configuration/entityTypes/Individual
Authorization: <access_token>
Content-Type: application/json

<new configuration definition for the Individual entity type>
```

Example response

The following examples show the response pattern for successful and error outcomes.

Successful response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created configuration definition>
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<BAD_REQUEST>
```

Update an object definition

Use the [Configuration API](#) to update the configuration definition for a specific type within a supported object type. This operation requires UPDATE access for the specified type and can update a definition only when inheritance is disabled for that type.

HTTP method and endpoint

Use the following HTTP method and endpoint path for the Configuration API request:

PUT {TenantURL}/configuration/{objectType}/{typeName}

Name	Required	Description
TenantURL	Yes	The base URL for your tenant.
objectType	Yes, for the general object type endpoint	The configuration object type that contains the definition to update. Supported values for this endpoint are entityTypes, groupTypes, relationTypes, graphTypes, interactionTypes, changeRequestTypes, and categoryTypes. This endpoint does not support sources.  Note: Only one changeRequestType can be configured at a time.
typeName	Yes	The name of the specific type definition to update.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes	application/json

Example request

The following example updates the **Individual** entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual
Authorization: <access_token>
Content-Type: application/json

<updated configuration definition for the Individual entity type>
```

Example response

The exact response payload depends on the requested object type and the updated definition. The following examples show the response pattern for successful and error outcomes.

Successful response

```
HTTP/1.1 200 OK
Content-Type: application/json

<updated configuration definition>
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<VALIDATION_ERROR or TYPE_IS_NOT_FOUND>
```

Delete an object definition

Use the [Configuration API](#) to delete the configuration definition for a specific type within a supported object type. This operation requires DELETE access for the specified type. You can delete the object definition only if it exists and is not in use.

HTTP method and endpoint

Use the following HTTP method and endpoint path to submit the request:

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
objectType	Yes, for the general object type endpoint	The configuration object type that contains the definition to delete. Supported values for this endpoint are entityTypes , groupTypes , relationTypes , graphTypes , interactionTypes , changeRequestTypes and categoryTypes . This endpoint does not support sources .
typeName	Yes	The name of the specific type definition to delete.

Request headers

Include the following header in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.

Example request

The following example deletes the **Individual** entity type definition.

```
DELETE {TenantURL}/configuration/entityTypes/Individual
Authorization: <access_token>
```

Example response

The following examples show the response pattern for successful and error outcomes.

Successful response

HTTP/1.1 204 No Content

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND>
```

Manage configuration sub-resources

Use the [Configuration API](#) to manage configuration sub-resources for entity types and relation types. These APIs support retrieval of sub-resource collections and individual definitions, as well as creation, update, and deletion for supported sub-resources.

Supported operations

The following operations are available:

- Cleanse configuration for entity types
- Survivorship groups for entity types
- Survivorship groups for relation types

- [Match groups for entity types](#)
- [Attributes for entity types](#)
- [Attributes for relation types](#)

All endpoints require the appropriate role-based access.

- **GET**: READ access
- **POST**: CREATE access, if supported for the sub-resource
- **PUT**: UPDATE access
- **DELETE**: DELETE access

Cleanse configuration for entity types

Use the [Configuration API](#) to retrieve, update, and delete the `cleanseConfig` sub-resource for an entity type. By default, GET responses include inherited configuration. To exclude inherited configuration, set the `inheritance` query parameter to `false`.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

GET retrieves the `cleanseConfig` definition for the specified entity type.

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/cleanseConfig?inheritance=true|false
```

PUT updates the `cleanseConfig` definition for the specified entity type.

```
PUT {TenantURL}/configuration/entityTypes/{entityTypeName}/cleanseConfig
```

DELETE deletes the `cleanseConfig` definition for the specified entity type.

```
DELETE {TenantURL}/configuration/entityTypes/{entityTypeName}/cleanseConfig
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL for your tenant.
entityTypeName	Yes	The name of the entity type whose <code>cleanseConfig</code> sub-resource you want to retrieve, update, or delete.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for PUT	<code>application/json</code>

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is

Example requests

The following example retrieves the `cleanseConfig` sub-resource for the `Individual` entity type and includes inherited configuration.

```
GET {TenantURL}/configuration/entityTypes/Individual/cleanseConfig?inheritance=true
Authorization: <access_token>
```

The following example updates the `cleanseConfig` sub-resource for the `Individual` entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual/cleanseConfig
Authorization: <access_token>
Content-Type: application/json

<updated cleanseConfig definition>
```

The following example deletes the `cleanseConfig` sub-resource for the `Individual` entity type.

```
DELETE {TenantURL}/configuration/entityTypes/Individual/cleanseConfig
Authorization: <access_token>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<cleanseConfig definition>
```

Successful PUT response

```
HTTP/1.1 200 OK
Content-Type: application/json

<updated cleanseConfig definition>
```

Successful DELETE response

HTTP/1.1 204 No Content

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND or VALIDATION_ERROR>
```

Survivorship groups for relation types

Use the [Configuration API](#) to retrieve survivorship groups for a relation type, retrieve a specific survivorship group, and create a new survivorship group. By default, GET responses include inherited

configuration. To exclude inherited configuration, set the `inheritance` query parameter to `false`.

HTTP methods and endpoints

```
GET {TenantURL}/configuration/relationTypes/{relationshipTypeName}/survivorshipGroups?inheritance=true|false
```

POST creates a new survivorship group definition for the specified relation type.

```
POST {TenantURL}/configuration/relationTypes/{relationshipTypeName}/survivorshipGroups/{groupUri}
```

PUT updates the specified survivorship group definition for the relation type.

```
PUT {TenantURL}/configuration/relationTypes/{relationshipTypeName}/survivorshipGroups/{groupUri}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
relationshipTypeName	Yes	The name of the relation type whose <code>survivorshipGroups</code> sub-resource you want to retrieve or update.
groupUri	Yes, for group-specific GET and POST	The URI of the survivorship group to retrieve or create.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for POST and PUT	<code>application/json</code>

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is included in the response. Accepted values are <code>true</code> and <code>false</code> . Default: <code>true</code> .

Example requests

The following example retrieves all `survivorshipGroups` for the `HasAddress` relation type and includes inherited configuration.

```
GET {TenantURL}/configuration/relationTypes/HasAddress/survivorshipGroups?inheritance=true
Authorization: <access_token>
```

The following example creates the **PrimaryAddressGroup** survivorship group for the **HasAddress** relation type.

```
<new survivorship group definition>
```

The following example updates the **PrimaryAddressGroup** survivorship group for the **HasAddress** relation type.

```
PUT {TenantURL}/configuration/relationTypes/HasAddress/survivorshipGroups/PrimaryAddressGroup
Authorization: <access_token>
Content-Type: application/json

<updated survivorship group definition>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<survivorship group definition or list of survivorship groups>
```

Successful POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created survivorship group definition>
```

Successful PUT response

```
HTTP/1.1 202 Accepted
Content-Type: application/json

<updated survivorship group definition>
```

Other error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND or BAD_REQUEST>
```

Survivorship groups for entity types

Use the [Configuration API](#) to retrieve survivorship groups for an entity type, retrieve a specific survivorship group, and create a new survivorship group. By default, GET responses include inherited configuration. To exclude inherited configuration, set the **inheritance** query parameter to **false**.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

GET retrieves all survivorship groups for the specified entity type or retrieves a specific survivorship group definition.

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/survivorshipGroups?inheritance=true|false
```

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/survivorshipGroups/{groupUri}?inheritance=true|false
```

[Reltio](#) [Products](#) **[Developer resources](#)** [Roles](#) [Applications](#) [Objectives](#)

```
POST {TenantURL}/configuration/entityTypes/{entityTypeName}/survivorshipGroups/{groupUri}
```

PUT updates the specified survivorship group definition for the entity type.

```
PUT {TenantURL}/configuration/entityTypes/{entityTypeName}/survivorshipGroups/{groupUri}
```

DELETE deletes the specified survivorship group definition for the entity type.

```
DELETE {TenantURL}/configuration/entityTypes/{entityTypeName}/survivorshipGroups/{groupUri}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
entityTypeName	Yes	The name of the entity type whose survivorshipGroups sub-resource you want to retrieve or update.
groupUri	Yes, for group-specific GET and POST	The URI of the survivorship group to retrieve or create.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for POST and PUT	application/json

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is included in the response. Accepted values are true and false . Default: true .

Example requests

The following example retrieves all **survivorshipGroups** for the **Individual** entity type and includes inherited configuration.

```
GET {TenantURL}/configuration/entityTypes/Individual/survivorshipGroups?inheritance=true
Authorization: <access_token>
```

The following example retrieves the **PrimaryNameGroup** survivorship group for the **Individual** entity type.

```
GET {TenantURL}/configuration/entityTypes/Individual/survivorshipGroups/PrimaryNameGroup?inheritance=true
Authorization: <access_token>
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

The following example creates the **PrimaryNameGroup** survivorship group for the **Individual** entity type.

```
POST {TenantURL}/configuration/entityTypes/Individual/survivorshipGroups/PrimaryNameGroup
Authorization: <access_token>
Content-Type: application/json

<new survivorship group definition>
```

The following example updates the **PrimaryNameGroup** survivorship group for the **Individual** entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual/survivorshipGroups/PrimaryNameGroup
Authorization: <access_token>
Content-Type: application/json

<updated survivorship group definition>
```

The following example deletes the **PrimaryNameGroup** survivorship group for the **Individual** entity type.

```
DELETE {TenantURL}/configuration/entityTypes/Individual/survivorshipGroups/PrimaryNameGroup
Authorization: <access_token>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<survivorship group definition or list of survivorship groups>
```

Successful POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created survivorship group definition>
```

Successful PUT response

```
HTTP/1.1 202 Accepted
Content-Type: application/json

<updated survivorship group definition>
```

Successful DELETE response

```
HTTP/1.1 200 OK
Content-Type: application/json

<deleted survivorship group response>
```

Error response

Match groups for entity types

Use the [Configuration API](#) to retrieve match groups for an entity type, retrieve a specific match group, create a new match group, update a match group, and delete a match group. By default, GET responses include inherited configuration. To exclude inherited configuration, set the **inheritance** query parameter to **false**.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

GET retrieves all match groups for the specified entity type or retrieves a specific match group definition.

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/matchGroups?inheritance=true|false
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/matchGroups/{matchGroupName}?
inheritance=true|false
```

POST creates a new match group definition for the specified entity type.

```
POST {TenantURL}/configuration/entityTypes/{entityTypeName}/matchGroups/{matchGroupName}
```

PUT updates the specified match group definition for the entity type.

```
PUT {TenantURL}/configuration/entityTypes/{entityTypeName}/matchGroups/{matchGroupName}
```

DELETE deletes the specified match group definition for the entity type.

```
DELETE {TenantURL}/configuration/entityTypes/{entityTypeName}/matchGroups/{matchGroupName}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
entityTypeName	Yes	The name of the entity type whose matchGroups sub-resource you want to retrieve or modify.
matchGroupName	Yes, for match-group-specific GET, POST, PUT, and DELETE	The name of the match group to retrieve, create, update, or delete.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for POST and PUT	application/json

Query parameters

The following table describes the query parameters.

	Reltio	Products	Developer resources	Roles	Applications	Objectives
inheritance	No, for GET		Determines whether inherited configuration from base types is included in the response. Accepted values are true and false . Default: true .			

Example requests

The following example retrieves all **matchGroups** for the **Individual** entity type and includes inherited configuration.

```
GET {TenantURL}/configuration/entityTypes/Individual/matchGroups?inheritance=true
Authorization: <access_token>
```

The following example retrieves the **ExactNameMatch** match group for the **Individual** entity type.

```
GET {TenantURL}/configuration/entityTypes/Individual/matchGroups/ExactNameMatch?inheritance=true
Authorization: <access_token>
```

The following example creates the **ExactNameMatch** match group for the **Individual** entity type.

```
POST {TenantURL}/configuration/entityTypes/Individual/matchGroups/ExactNameMatch
Authorization: <access_token>
Content-Type: application/json

<new match group definition>
```

The following example updates the **ExactNameMatch** match group for the **Individual** entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual/matchGroups/ExactNameMatch
Authorization: <access_token>
Content-Type: application/json

<updated match group definition>
```

The following example deletes the **ExactNameMatch** match group for the **Individual** entity type.

```
DELETE {TenantURL}/configuration/entityTypes/Individual/matchGroups/ExactNameMatch
Authorization: <access_token>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<match group definition or list of match groups>
```

Successful POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created match group definition>
```

Successful PUT response

```
HTTP/1.1 200 OK
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

Successful DELETE response

```
HTTP/1.1 204
{
  "successful": true
}
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND, BAD_REQUEST, or VALIDATION_ERROR>
```

Attributes for entity types

Use the [Configuration API](#) to retrieve attributes for an entity type, retrieve a specific attribute, create a new attribute, update an attribute, and delete an attribute. By default, GET responses include inherited configuration. To exclude inherited configuration, set the **inheritance** query parameter to **false**.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

GET retrieves all attributes for the specified entity type or retrieves a specific attribute definition.

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/attributes?inheritance=true|false
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/attributes/{attributeUri}?inheritance=true|false
```

POST creates a new attribute definition for the specified entity type.

```
POST {TenantURL}/configuration/entityTypes/{entityTypeName}/attributes/{attributeUri}
```

PUT updates the specified attribute definition for the entity type.

```
PUT {TenantURL}/configuration/entityTypes/{entityTypeName}/attributes/{attributeUri}
```

DELETE deletes the specified attribute definition for the entity type.

```
DELETE {TenantURL}/configuration/entityTypes/{entityTypeName}/attributes/{attributeUri}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL for your tenant.
attributeUri	Yes, for attribute-specific GET, POST, PUT, and DELETE	The URI of the attribute to retrieve, create, update, or delete.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for POST and PUT	application/json

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is included in the response. Accepted values are true and false. Default: true.

Example requests

The following example retrieves all attributes for the Individual entity type and includes inherited configuration.

```
GET {TenantURL}/configuration/entityTypes/Individual/attributes?inheritance=true
Authorization: <access_token>
```

The following example retrieves the Name attribute for the Individual entity type.

```
GET {TenantURL}/configuration/entityTypes/Individual/attributes/Name?inheritance=true
Authorization: <access_token>
```

The following example creates the Name attribute for the Individual entity type.

```
POST {TenantURL}/configuration/entityTypes/Individual/attributes/Name
Authorization: <access_token>
Content-Type: application/json

<new attribute definition>
```

The following example updates the Name attribute for the Individual entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual/attributes/Name
Authorization: <access_token>
Content-Type: application/json

<updated attribute definition>
```

The following example deletes the **Name** attribute for the **Individual** entity type.

```
DELETE {TenantURL}/configuration/entityTypes/Individual/attributes/Name
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<attribute definition or list of attributes>
```

Successful POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created attribute definition>
```

Successful PUT response

```
HTTP/1.1 200 OK
Content-Type: application/json

<updated attribute definition>
```

Successful DELETE response

```
HTTP/1.1 204 No Content
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND, BAD_REQUEST, or VALIDATION_ERROR>
```

Attributes for relation types

Use the [Configuration API](#) to retrieve attributes for a relation type, retrieve a specific attribute, create a new attribute, update an attribute, and delete an attribute. By default, GET responses include inherited configuration. To exclude inherited configuration, set the **inheritance** query parameter to **false**.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

GET retrieves all attributes for the specified relation type or retrieves a specific attribute definition.

```
GET {TenantURL}/configuration/relationTypes/{relationshipTypeName}/attributes?inheritance=true|false
GET {TenantURL}/configuration/relationTypes/{relationshipTypeName}/attributes/{attributeUri}?inheritance=true|false
```

POST creates a new attribute definition for the specified relation type.

```
POST {TenantURL}/configuration/relationTypes/{relationshipTypeName}/attributes/{attributeUri}
```

PUT updates the specified attribute definition for the relation type.

```
PUT {TenantURL}/configuration/relationTypes/{relationshipTypeName}/attributes/{attributeUri}
```

[Reltio](#) [Products](#) **[Developer resources](#)** [Roles](#) [Applications](#) [Objectives](#)

DELETE deletes the specified attribute definition for the relation type.

```
DELETE {TenantURL}/configuration/relationTypes/{relationshipTypeName}/attributes/{attributeUri}
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL for your tenant.
relationshipTypeName	Yes	The name of the relation type whose attributes sub-resource you want to retrieve or modify.
attributeUri	Yes, for attribute-specific GET, POST, PUT, and DELETE	The URI of the attribute to retrieve, create, update, or delete.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for POST and PUT	application/json

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is included in the response. Accepted values are true and false . Default: true .

Example requests

The following example retrieves all **attributes** for the **HasAddress** relation type and includes inherited configuration.

```
GET {TenantURL}/configuration/relationTypes/HasAddress/attributes?inheritance=true
Authorization: <access_token>
```

The following example retrieves the **AddressType** attribute for the **HasAddress** relation type.

```
GET {TenantURL}/configuration/relationTypes/HasAddress/attributes/AddressType?inheritance=true
Authorization: <access_token>
```

The following example creates the **AddressType** attribute for the **HasAddress** relation type.

```
POST {TenantURL}/configuration/relationTypes/HasAddress/attributes/AddressType
Authorization: <access_token>
Content-Type: application/json
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

The following example updates the **AddressType** attribute for the **HasAddress** relation type.

```
PUT {TenantURL}/configuration/relationTypes/HasAddress/attributes/AddressType
Authorization: <access_token>
Content-Type: application/json

<updated attribute definition>
```

The following example deletes the **AddressType** attribute for the **HasAddress** relation type.

```
DELETE {TenantURL}/configuration/relationTypes/HasAddress/attributes/AddressType
Authorization: <access_token>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<attribute definition or list of attributes>
```

Successful POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

<created attribute definition>
```

Successful PUT response

```
HTTP/1.1 200 OK
Content-Type: application/json

<updated attribute definition>
```

Successful DELETE response

```
HTTP/1.1 204 No Content
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND, BAD_REQUEST, or VALIDATION_ERROR>
```

Surrogate crosswalks for entity types

Use the [Configuration API](#) to retrieve, update, and delete the **surrogateCrosswalks** sub-resource for an entity type. By default, GET responses include inherited configuration. To exclude inherited configuration, set the **inheritance** query parameter to **false**.

HTTP methods and endpoints

Use one of the following HTTP methods and endpoint paths to submit the request:

```
GET {TenantURL}/configuration/entityTypes/{entityTypeName}/surrogateCrosswalks?inheritance=true|false
```

PUT updates the surrogateCrosswalks definition for the specified entity type.

```
PUT {TenantURL}/configuration/entityTypes/{entityTypeName}/surrogateCrosswalks
```

DELETE deletes the surrogateCrosswalks definition for the specified entity type.

```
DELETE {TenantURL}/configuration/entityTypes/{entityTypeName}/surrogateCrosswalks
```

The following table describes the endpoint path parameters.

Name	Required	Description
TenantURL	Yes	The base URL of your tenant.
entityTypeName	Yes	The name of the entity type whose surrogateCrosswalks sub-resource you want to retrieve, update, or delete.

Request headers

Include the following headers in the request.

Name	Required	Description
Authorization	Yes	Access token used to authenticate the tenant request.
Content-Type	Yes, for PUT	application/json

Query parameters

The following table describes the query parameters.

Name	Required	Description
inheritance	No, for GET	Determines whether inherited configuration from base types is included in the response. Accepted values are true and false. Default: true.

Example requests

The following example retrieves the surrogateCrosswalks sub-resource for the Individual entity type and includes inherited configuration.

```
GET {TenantURL}/configuration/entityTypes/Individual/surrogateCrosswalks?inheritance=true
Authorization: <access_token>
```

The following example updates the surrogateCrosswalks sub-resource for the Individual entity type.

```
PUT {TenantURL}/configuration/entityTypes/Individual/surrogateCrosswalks
Authorization: <access_token>
Content-Type: application/json
```

```
<updated surrogateCrosswalks definition>
```

```
DELETE {TenantURL}/configuration/entityTypes/Individual/surrogateCrosswalks
Authorization: <access_token>
```

Example responses

The following examples show the response pattern for successful and error outcomes.

Successful GET response

```
HTTP/1.1 200 OK
Content-Type: application/json

<surrogateCrosswalks definition>
```

Successful PUT response

```
HTTP/1.1 200 OK
Content-Type: application/json

<updated surrogateCrosswalks definition>
```

Successful DELETE response

```
HTTP/1.1 204 No Content
```

Error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

<TYPE_IS_NOT_FOUND or VALIDATION_ERROR>
```

Role-Based Security

Related concepts

[Metadata Security](#)

[Data Security](#)

Metadata Security

The Reltio Data Cloud platform supports role-based access control (RBAC) with its metadata security framework. Each role has a list of resources with operations that users assigned to the role can do on each resource. Security can be set at an entity/relationship type level or for individual attributes.

 **Note:** For a newly created tenant, metadata security is disabled. It is enabled when any permissions configuration (even an empty configuration) is applied to the tenant. You may require metadata permissions for performing some of the tasks through Hub, such as crosswalk deletion.

Get Tenant Configuration

The tenant configuration API request supports option **showAccess**. If this option is enabled, the business model configuration of a tenant is returned along with access level specifications for all entity types, relationship types, and attributes.

```
GET {TenantURL}/configuration?options=showAccess
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
```

Response

```
...
"attributes": [
  {
    "uri": "configuration/entityTypes/HCP/attributes/Prefix",
    "label": "Prefix",
    "name": "Prefix",
    "description": "",
    "type": "String",
    "hidden": false,
    "important": false,
    "faceted": true,
    "searchable": true,
    "attributeOrdering": {
      "orderingStrategy": "LUD"
    },
    "access": ["CREATE", "UPDATE", "READ"]
  },
  {
    "uri": "configuration/entityTypes/HCP/attributes/Name",
    "label": "Full Name",
    "name": "Name",
    "description": "Name",
    "type": "String",
    "hidden": false,
    "important": false,
    "searchable": true,
    "attributeOrdering": {
      "orderingStrategy": "LUD"
    },
    "access": ["READ"]
  },
  ...
]
```

Set Permissions

Request

```
POST {env_uri}/relio/permissions/{tenant}
```

This request sets permissions for all specified URIs and roles. All permissions for each specified URI and role are updated recursively.

Tip: An example for {env_uri}/relio is <https://abcd-01.relio.com/relio>.

Parameters

Parameter	Name	Required	Details
Headers	Authorization	Yes	Information about authentication access token in format Bearer <accessToken> (see details in Authentication API).
	Content-Type	Yes	Must be Content-Type: application/json .
Body		Yes	JSON Array with new permission records. Each record must have "uri", "role" and a list of access types. This list can contain one or several values or can be empty. Possible access types: CREATE, READ, UPDATE, DELETE, MERGE, UNMERGE, INITIATE_CHANGE_REQUEST, ACCEPT_CHANGE_REQUEST.

Response

The response contains all permissions for all the URIs from the request (same as for `_get` request for those URIs).

including the to-be-removed role. First, you must delete the entire role configuration for the user by using the **Delete configuration** option and then resend the **Post** request to assign only the required roles to the user.

Setting Permissions

Request

```
POST {env_uri}/reltio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  {
    "uri": "configuration/entityTypes",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["READ"]
      },
      {
        "role": "UI_USER",
        "access": ["READ"]
      }
    ]
  },
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      }
    ]
  }
]
```

Response

```
[
  {
    "uri": "configuration/entityTypes",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["READ"]
      },
      {
        "role": "UI_USER",
        "access": ["READ"]
      }
    ]
  },
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      }
    ]
  }
]
```

 **Note:** If you define multiple permissions for the same role and the same access type with different filters, only the last one is retained. Earlier definitions are overwritten. Avoid defining conflicting filtered permissions for the same role and access combination.

Adding More Permissions

Request

```
POST {env_uri}/reltio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  {
```



```
"uri": "configuration/entityTypes",
"permissions": [
  {
    "role": "DATA_STEWARD",
    "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
  }
]
```

```
{
  "uri": "configuration/entityTypes",
  "permissions": [
    {
      "role": "APL_USER",
      "access": ["READ"]
    },
    {
      "role": "DATA_STEWARD",
      "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
    },
    {
      "role": "UL_USER",
      "access": ["READ"]
    }
  ]
}
```

Response

```
[
  {
    "uri": "configuration/entityTypes",
    "permissions": [
      {
        "role": "APL_USER",
        "access": ["READ"]
      },
      {
        "role": "DATA_STEWARD",
        "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
      },
      {
        "role": "UL_USER",
        "access": ["READ"]
      }
    ]
  }
]
```

Update Permissions

You can update existing permissions or add new ones for specified URIs and roles without replacing the entire configuration.

Request

```
PUT {env_uri}/relio/permissions/{tenant}
```

Parameters

Parameter	Name	Required	Details
Headers	Authorization	Yes	Bearer access token. See Authentication API .
	Content-Type	Yes	Must be Content-Type: application/json .
Body		Yes	JSON array of permission records. Each record must include: uri: URI of the configuration object permissions: role, optional filter, and list of access types access: array of operations, such as CREATE, READ, UPDATE, DELETE, MERGE, UNMERGE, INITIATE_CHANGE_REQUEST, ACCEPT_CHANGE_REQUEST

Response

Returns all updated permissions for the specified URIs.

Updating Permissions for URIs

Example request

```
PUT {env_uri}/relio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  {
    "permissions": [
      {
        "access": ["UPDATE"],
        "role": "API_USER"
      }
    ]
  }
]
```

```
    },
    "uri": "configuration/entityTypes"
  },
  {
    "permissions": [
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

```
    ]
  },
  "uri": "configuration/entityTypes/Location"
}
]
```

Example response

```
[
  {
    "permissions": [
      {
        "access": ["UPDATE"],
        "role": "API_USER"
      },
      {
        "access": ["READ"],
        "role": "UI_USER"
      }
    ],
    "uri": "configuration/entityTypes"
  },
  {
    "permissions": [
      {
        "access": ["CREATE","READ","UPDATE"],
        "role": "API_USER"
      }
    ],
    "uri": "configuration/entityTypes/HCP"
  },
  {
    "permissions": [
      {
        "access": ["READ"],
        "filter": "containsWordStartingWith(attributes,U)",
        "role": "API_USER"
      }
    ],
    "uri": "configuration/entityTypes/Location"
  }
]
```

Get Permissions for URIs

Returns permissions for specified URIs (with inheritance of URIs) and roles.

Request

```
POST {env_uri}/reltio/permissions/{tenant}/_get
```

Contains a list of URIs to get information about allowed operations for the specified user roles.

 **Note:** If no query parameter is passed, then all user roles are considered.

Parameter	Name	Required	Details
-----------	------	----------	---------

Reltio Products **Developer resources** Roles Applications Objectives

			Authentication API).
	Content-Type	Yes	Must be Content-Type: application/json.
Query	roles	No	List of roles. Default: all roles which have any privileges for an URI.
Body		Yes	List of valid URIs. They can have corresponding permissions records or not. In the latter case, permissions will be obtained considering URIs inheritance. For example, "configuration/entityTypes/HCP" extends "configuration/entityTypes".

Response

For each URI from an input list, the response contains information about all privileges for the specified roles.

Getting All Permissions for a List of URIs

Request

```
POST {env_uri}/reltio/permissions/{tenant}/_get
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  "configuration/entityTypes/HCP",
  "configuration/entityTypes/Location"
]
```

Response

```
[
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      },
      {
        "role": "DATA_STEWARD",
        "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
      },
      {
        "role": "UI_USER",
        "access": ["READ"]
      }
    ]
  },
  {
    "uri": "configuration/entityTypes/Location",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["READ"]
      },
      {
        "role": "DATA_STEWARD",
        "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
      },
      {
        "role": "UI_USER",
        "access": ["READ"]
      }
    ]
  }
]
```

Getting Permissions for Specific Roles

Request

```
POST {env_uri}/relio/permissions/{tenant}/_get?roles=APL_USER
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
```



Response

```
[
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "APL_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      }
    ]
  },
  {
    "uri": "configuration/entityTypes/Location",
    "permissions": [
      {
        "role": "APL_USER",
        "access": ["READ"]
      }
    ]
  }
]
```

Get Configuration

Returns full permissions configuration.

Request

```
GET {env_uri}/relio/permissions/{tenant}
```

Parameters

Parameter	Name	Required	Details
Headers	Authorization	Yes	Information about authentication access token in format Bearer <accessToken> (see details in Authentication API).
	Content-Type	Yes	Must be Content-Type: application/json .
Query	roles	No	List of roles. Default: all roles which have any privileges for an URI.

Response

Returns all permissions configuration records for a tenant. If some user roles are specified as a query parameter, irrelevant records will be excluded from the response.

Getting Full Configuration

Request

```
GET {env_uri}/relio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
```

Response

```
[
  {
    "uri": "configuration/entityTypes",
    "permissions": [
      {
        "role": "APL_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      }
    ]
  }
]
```

```
    },
    {
      "role": "DATA_STEWARD",
      "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
    }
  ],
}
```

```
    ],
  },
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "API_USER",
        "access": ["CREATE", "READ", "UPDATE"]
      }
    ]
  }
]
```

Getting All Records for Specific Roles

Request

```
GET {env_uri}/reltio/permissions/{tenant}?roles=UI_USER,DATA_STEWARD
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
```

Response

```
[
  {
    "uri": "configuration/entityTypes",
    "permissions": [
      {
        "role": "DATA_STEWARD",
        "access": ["CREATE", "READ", "UPDATE", "DELETE", "MERGE", "UNMERGE"]
      },
      {
        "role": "UI_USER",
        "access": ["READ"]
      }
    ]
  }
]
```

Delete Configuration

You can remove all tenant permissions or delete only specific parts of the configuration.

Request

```
DELETE {env_uri}/reltio/permissions/{tenant}
```

Parameters

Parameter	Name	Required	Details
Headers	Authorization	Yes	Information about authentication access token in format Bearer <accessToken> (see details in Authentication API).
	Content-Type	Yes	Must be Content-Type: application/json .
Body		Optional	If NO body is provided: the entire security configuration is removed and metadata security is disabled (see example 1 below). If a body is provided: only the matching uri, role, and optional filter are removed (see example 2 below).

Response

Returns the status of the request execution.

Example 1: Remove entire configuration

```
DELETE {env_uri}/reltio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
```

Response

```
{
  "status": "success"
}
```

Example 2: Remove role from a specific URI with filter

Request

```
DELETE {env_uri}/reltio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  {
    "permissions": [
      {
        "role": "API_USER",
        "filter": "containsWordStartingWith(attributes,U)"
      }
    ],
    "uri": "configuration/entityTypes/Location"
  }
]
```

Response

```
{
  "status": "success"
}
```

Check User Permissions

Returns available permissions for roles of a current user and for specified URIs. If a user has more than one role, then they will be able to do the maximum from their roles.

 **Note:** The API is available for all users and is designed to check permissions for their roles.

Request

```
POST {env_uri}/reltio/permissions/{tenant}/_check
```

The request contains a list of URIs to check if they are allowed for a user.

Parameters

Parameter	Name	Required	Details
Headers	Authorization	Yes	Information about authentication access token in format Bearer <accessToken> (see details in Authentication API).
	Content-Type	Yes	Must be Content-Type: application/json .
Body		No	List of valid URIs. Permissions will be obtained considering the URIs inheritance and all roles of a user.

For each URI from the input list, contains a set of allowed access types (this list can be empty).

```
POST {env_uri}/reltio/permissions/{tenant}/_check
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Role: API_USER
Body:
[
  "configuration/entityTypes/HCP",
  "configuration/entityTypes/Location"
]
```

Response

```
{
  "configuration/entityTypes/HCP": ["CREATE", "UPDATE", "READ"],
  "configuration/entityTypes/Location": ["READ"]
}
```

Hide Attributes in Reltio UI

Suppose we need to hide some attributes in the UI by using the role-based metadata security. For this purpose, we must post permissions for the attributes with empty access.

Hiding Attributes by Metadata Security

The example below illustrates how to hide the first name of an HCP for role ROLE_HEART_READ.

Request

```
POST {env_uri}/reltio/permissions/{tenant}
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  {
    "uri": "configuration/entityTypes/HCP/attributes/Name",
    "permissions": [
      {
        "role": "ROLE_HEART_READ",
        "access": []
      }
    ]
  }
]
```

 **Note:** Metadata security is not compatible with Analytics attributes. You cannot apply the Metadata security rules to analytics attributes.

Related concepts

[Role-Based Security](#)

[Data Security](#)

[GET Configuration](#)

Access restrictions to data sources

Any customer can have their data provided from different sources. This data is personal and sometimes sensitive, so you must be able to create restrictions that prevent all users from accessing all the data from all data sources. You might want to allow a specific set of users to only access a specific data source.

In metadata security, this is possible by using the slice filter, which slices the data based on the sources. When you create a role, add the slice filter specifying the data sources that role holders can access. When you assign this role to a user, the filter restricts their access to the specified data sources.

This example shows how to configure the role ROLE_TWIT_AND_FB_SLICE_READ to limit user access to the Twitter and FaceBook source systems. When this role is assigned to a user, they can only access these two data sources, data from any other sources is filtered out.

```
{ "role" : "ROLE_TWIT_AND_FB_SLICE_READ", "filter" : "slice(sourceSystems,FB) AND slice(sourceSystems,TWITTER)",  
  "access" : ["READ"] }
```

Slice filter usage conditions

Consider these points when you configure the slice filter:

- Use the Entities API GET method to slice an entity, for example **GET /{tenant}/entities/{id}**.
- The slice filter works only for entities. It doesn't work for relationships.
- The slice filter applies only to READ permissions and takes priority over other filters. If you apply the filter to other permissions, such as UPDATE or MERGE, the API returns a `FilterException` error and doesn't complete the process, even if the user has been assigned another role with other permissions.

```
{ "role" : "ROLE_FB_SLICE_READ", "filter" : "slice(sourceSystems,FB)", "access" : ["UPDATE"] }
```

- The slice filter works only with `sourceSystems`. If you apply the filter to other values, the API returns a `FilterException` error and doesn't complete the process.

```
{ "role" : "ROLE_FB_SLICE_INVALID_READ", "filter" : "slice(sourceSystems,FB) AND slice(type,  
'configuration/entityTypes/ProductItem)", "access" : ["READ"] }
```

- Use commas to separate multiple source names in the slice filter.

```
{ "role" : "ROLE_TWIT_AND_FB_SLICE_READ", "filter" : "slice(sourceSystems,FB,TWITTER)", "access" : ["READ"] }
```

- Use AND and OR conditional operators to apply more than one slice filters to a role. AND and OR conditional operators behave the same for the slice filter.

```
{ "role" : "ROLE_TWIT_AND_FB_SLICE_READ", "filter" : "slice(sourceSystems,FB) AND slice(sourceSystems,TWITTER)",  
  "access" : ["READ"] }  
{ "role" : "ROLE_TWIT_OR_FB_SLICE_READ", "filter" : "slice(sourceSystems,FB) OR slice(sourceSystems,TWITTER)",  
  "access" : ["READ"] }
```

- Use NOT conditional operators to restrict access to a specific source system. NOT conditional operators take priority over any other operators specified in the filter.

The user who's been assigned these roles can't access data from the TWITTER and FB sources.

```
{ "role" : "ROLE_TWIT_NOT_AND_FB_SLICE_READ", "filter" : "not slice(sourceSystems,FB) AND not  
slice(sourceSystems,TWITTER)", "access" : ["READ"] }  
{ "role" : "ROLE_TWIT_NOT_AND_FB_SLICE_AND_FB_READ", "filter" : "not slice(sourceSystems,FB) AND not  
slice(sourceSystems,TWITTER) AND slice(sourceSystems,'FB')", "access" : ["READ"] }
```

- If a user has been assigned two different roles with different slice filters, the filter permissions don't overlap. In this example, if the same user is assigned the `ROLE_TWIT_READ` role and the `ROLE_FB_READ` role, they can read data from both Twitter and Facebook, but no other data sources.

```
{ "role" : "ROLE_TWIT_READ", "filter" : "slice(sourceSystems,TWITTER)", "access" : ["READ"] }  
{ "role" : "ROLE_FB_READ", "filter" : "slice(sourceSystems,FB)", "access" : ["READ"] }
```


- Use AND and OR conditional operators to combine the slice filter with other filters.

The slice filter always returns a value of true. If you combine the slice filter with other filters, evaluate the request carefully to ensure you get the desired result. In this example, the slice filter is true but

Reltio Products **Developer resources** Roles Applications Objectives

```
{ "role" : "ROLE_TWIT_READ", "filter" : "slice(sourceSystems,TWITTER) AND equals(type, 'configuration/entityTypes/ProductItem)', "access" : ["READ"] }
```

Data Security

Security rules can be applied for a subset of entities defined by a specific condition. Such conditions are represented by filters, similar to the filters in *Entity Search*.

 **Note:** Data security is disabled for a newly created tenant. It is enabled when any permissions configuration, even an empty one, is applied to the tenant.

Get Entity Permissions

Requests which return entities support option **showAccess**. If this option is enabled, the entities are returned along with access level specifications for the entities and reference objects (both reference entities and reference relations).

Getting Entities with Access

Request

```
GET {TenantURL}/entities/00005KL?options=showAccess
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
```

Response

```
{
  "uri": "entities/00005KL",
  "type": "configuration/entityTypes/HCP",
  ...
  "attributes": {
    "Employment": [
      {
        "label": "",
        "relationshipLabel": "Pediatrician",
        "value": {
          "Title": [
            {
              "type": "configuration/relationTypes/Employment/attributes/Title",
              "ov": true,
              "value": "Pediatrician",
              "uri": "entities/00005KL/attributes/Employment/00006OP/Title/1mtNpZ"
            }
          ]
        }
      },
      ...
    ],
    ...
  },
  "refEntity": {
    ...
    "type": "configuration/entityTypes/Organization",
    "objectURI": "entities/00009ab",
    "access": ["READ"]
  },
  "refRelation": {
    ...
    "objectURI": "relations/00006OP",
    "access": []
  },
  ...
},
...
"label": " ",
"access": ["READ", "CREATE", "UPDATE"]
}
```

Configuration API

Data security is configured together with metadata security. Both are similar and use the same endpoints. A data security configuration record has an additional **filter** field. The specified privileges are applied for all entities that are accepted by a filter.

Metadata security evaluates all values of an attribute. It does not consider Operational Value (OV), pinned, or ignored flags when applying filters.

- When Metadata Security filters reference attributes that use an RDM lookup type, don't share the same lookup type across multiple entity objects. Because lookup types are evaluated globally, filtering on one object can affect the lookup values displayed on another object. This behavior is a known limitation.

Data and Metadata Security Records

```
[
  {
    "uri": "configuration/entityTypes/HCP",
    "permissions": [
      {
        "role": "API_USER1",
        "access": ["READ"]
      },
      {
        "role": "API_USER2",
        "filter": "startsWith(attributes.FirstName, 'M')",
        "access": ["READ", "CREATE", "UPDATE"]
      }
    ]
  },
  {
    "uri": "configuration/entityTypes/Location",
    "permissions": [
      {
        "role": "API_USER1",
        "filter": "startsWith(attributes.City, 'New York')",
        "access": ["READ"]
      },
      {
        "role": "API_USER2",
        "access": ["READ", "CREATE", "UPDATE"]
      }
    ]
  },
  {
    "uri": "configuration/relationTypes/HasAddress",
    "permissions": [
      {
        "role": "API_USER1",
        "access": ["READ"]
      },
      {
        "role": "API_USER2",
        "access": ["READ", "CREATE", "UPDATE"]
      }
    ]
  }
]
```

Get Roles by Object URIs

This request is used to process specified URIs and return all available permissions for users, who are assigned the permission for an object. This is applicable for entities and Data Change Requests.

 **Note:** For **changeRequest**, only roles for **READ** and **ACCEPT_CHANGE_REQUEST** access are returned.

Request

```
POST {env_uri}/reltio/permissions/{tenant}/_getAccessRoles
```

Contains a list of URIs to get information about allowed operations and accepted users.

Parameter	Name	Required	Details
-----------	------	----------	---------

Reltio Products **Developer resources** Roles Applications Objectives

			Authentication API .
	Content-Type	Yes	Must be Content-Type: application/json .
Query	accessTypes	No	List of access types for which we want to obtain roles. Default: all access types.
Body		Yes	List of valid URIs. The operation can process only entities and data change requests. For example, <i>entities/0290P2m</i> or <i>changeRequests/00010rp</i> .

Response

For each URI from the input list, the response contains information about all privileges with all roles. The information is organized as a list of triplets with the following parts:

- Object URI: can be a position in the configuration tree or the URI of a particular object in the system (entity, relation, attribute value).
- Access: access type required for this object (directly requested or implied). Possible values: **CREATE**, **READ**, **UPDATE**, **DELETE**, **MERGE**, **UNMERGE**, **INITIATE_CHANGE_REQUEST**, **ACCEPT_CHANGE_REQUEST**.
- Roles: set of roles that are configured to have such access type to this object in this tenant.

There is an additional field in each list record: **allPermitted**. When the **accessRoles** section is empty, the value of the **allPermitted** field must be analyzed: if it is **true**, then any role has all permissions, otherwise there are no roles which have any permission on that particular object.

 **Note:** There are a few points to bear in mind when placing Data Change Request as a subject to get the required roles. Change requests cannot be specified as a configuration tree. **ACCEPT_CHANGE_REQUEST** are granted for entity types. Hence, the result set generated for each DCR contains all its change items represented by the URIs of the entity or the attribute specified in each change item. Every such item contains the **changeRequestUri** field to be able to associate objects from the request with objects from the response. **ObjectUri** may contain the URI of the DCR itself only if all the roles have full access to its content (for example, when security is off). Also, if access type **ACCEPT_CHANGE_REQUEST** is requested for a DCR, it is automatically converted to **ACCEPT_CHANGE_REQUEST**, **READ**, **CREATE**, **UPDATE**, **DELETE**, and **MERGE**.

Getting Roles for DCR

This example illustrates getting roles for a data change request from a tenant with no metadata security.

Request

```
POST {env_uri}/reltio/permissions/{tenant}/_getAccessRoles?accessTypes=ACCEPT_CHANGE_REQUEST
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  "changeRequests/0bfdIEy"
]
```

Response

```
[
  {
    "objectUri": "changeRequests/0bfdIEy",
    "changeRequestUri": "changeRequests/0bfdIEy",
    "allPermitted": true
  }
]
```

Getting Roles for Entity

This example illustrates getting roles for an entity when there are roles only for specific permissions.

Request

```
POST {env_uri}/relio/permissions/{tenant}/_getAccessRoles?accessTypes=READ,MERGE
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
```

[Reltio](#) [Products](#) [Developer resources](#) [Roles](#) [Applications](#) [Objectives](#)

Response

```
[
  {
    "objectUri": "entities/00005KL",
    "allPermitted": false,
    "accessRoles": [
      {
        "access": "READ",
        "roles": [
          "API_USER2",
          "API_USER1"
        ]
      },
      {
        "access": "MERGE",
        "roles": [
          "API_USER2",
          "API_USER1"
        ]
      }
    ]
  }
]
```

Getting Roles for DCR with Change Items for Entity and Attributes

Let's assume that change request *changeRequests/0Yd3jJw* contains three change items: one of type **CREATE_ENTITY** and two of type **INSERT_ATTRIBUTE**. The first change item **CREATE_ENTITY** is about the first entity with ID *entities/0Bymuem* and the other two change items are about an already existing entity with ID *entities/00006Mq*. **_getAccessRoles** returns four sections (triplets): one for the **CREATE_ENTITY** change item and three for the rest two ones. Among those three sections we can find a section with required permissions for *entities/00006Mq* itself because attributes manipulations require some permissions on the root object besides attributes.

 **Note:** With the **applyDcrWithoutPermissions** flag set to true in the configuration, you don't need UPDATE, CREATE, DELETE, or MERGE privileges on the objects to accept a change request, only the READ privilege. For more information, see topic [DCR Security](#)

Request

```
POST {env_uri}/relio/permissions/{tenant}/_getAccessRoles?accessTypes=ACCEPT_CHANGE_REQUEST
Headers: Authorization: Bearer <Access-Token>, Content-Type: application/json
Body:
[
  "changeRequests/0Yd3jJw"
]
```

Response

```
[
  {
    "objectUri": "entities/0Bymuem",
    "changeRequestUri": "changeRequests/0Yd3jJw",
    "allPermitted": false,
    "accessRoles": [
      {
        "access": "READ",
        "roles": [
          "UI_USER",
          "API_USER",
          "ROLE_US"
        ]
      },
      {
        "access": "CREATE",
        "roles": [
          "ROLE_US"
        ]
      },
      {
        "access": "ACCEPT_CHANGE_REQUEST",
        "roles": [
```

```
        "ROLE_US"
      ]
    },
    {
      "access": "MERGE",
    }
  ]
}
```

```
    ]
  },
  {
    "objectUri": "entities/00006Mq",
    "changeRequestUri": "changeRequests/0Yd3jJw",
    "allPermitted": false,
    "accessRoles": [
      {
        "access": "READ",
        "roles": [
          "UI_USER",
          "API_USER",
          "ROLE_US"
        ]
      },
      {
        "access": "ACCEPT_CHANGE_REQUEST",
        "roles": [
          "ROLE_US"
        ]
      },
      {
        "access": "MERGE",
        "roles": [
          "ROLE_US"
        ]
      },
      {
        "access": "UPDATE",
        "roles": [
          "ROLE_US"
        ]
      }
    ]
  },
  {
    "objectUri": "entities/00006Mq/attributes/FirstName/1EqLlaWG",
    "changeRequestUri": "changeRequests/0Yd3jJw",
    "allPermitted": false,
    "accessRoles": [
      {
        "access": "READ",
        "roles": [
          "UI_USER",
          "API_USER"
        ]
      },
      {
        "access": "CREATE",
        "roles": [
          "ROLE_US"
        ]
      },
      {
        "access": "ACCEPT_CHANGE_REQUEST",
        "roles": [
          "ROLE_US"
        ]
      }
    ]
  },
  {
    "objectUri": "entities/00006Mq/attributes/FirstName/1EqLlemW",
    "changeRequestUri": "changeRequests/0Yd3jJw",
    "allPermitted": false,
    "accessRoles": [
      {
        "access": "READ",
        "roles": [
          "UI_USER",
          "API_USER"
        ]
      },
      {
        "access": "CREATE",
        "roles": [
          "ROLE_US"
        ]
      },
      {
        "access": "ACCEPT_CHANGE_REQUEST",
        "roles": [
          "ROLE_US"
        ]
      }
    ]
  }
]
```

OV Rulesets Per Role

You can associate an operational value (OV) ruleset with the following:

- User Role - You can associate an arbitrary role with a ruleset. If a user has multiple roles with rulesets assigned but does not have an associated ruleset, then the ruleset that comes first in the configuration is the winner.
- User - If an OV ruleset is assigned to a user, it has higher precedence compared to an OV ruleset assigned to a user role.

If a user does not have any associated rulesets (by role or user), then the default ruleset is used.

Tenant Configuration

To activate assigning of a ruleset to a user or user role, create some non-default **survivorshipGroups** and add a mapping between user roles or a set of roles and the created groups as follows:

```
"PUT": {"TenantURL"}"/rulesetsConfiguration
BODY": {
  "configuration/entityTypes/HCP": {
    ...
    "survivorshipGroups": [
      {
        "uri": "configuration/entityTypes/Organization/survivorshipGroups/default",
        "Uri": "configuration/entityTypes/HCP/survivorshipGroups/group_test_1": [
          "ROLE_TEST_1"
        ]
      },
      "configuration/entityTypes/HCP/survivorshipGroups/group_test_2": [
        "ROLE_TEST_2",
        "ROLE_TEST_3"
      ],
      "mapping": [
        {
          "attribute": "configuration/entityTypes/Organization/attributes/Address",
          "survivorshipStrategy": "Aggregation"
        },
        {
          "attribute": "configuration/entityTypes/Organization/attributes/Identifiers",
          "survivorshipStrategy": "Aggregation"
        }
      ]
    }
  }
  ...
}
```

The mapping process has the following restrictions:

- You cannot add a mapping to a non-existing group.
- You cannot add an empty mapping.

If the request is successful, the response contains a list of mappings.

Current Rulesets

The current rulesets display the survivorship groups and roles that connect these survivorship groups to the current user. Use the following request to get the current rulesets:

```
GET: {TenantURL}/rulesetsConfiguration/currentRulesets

RESPONSE:
{
  "configuration/entityTypes/HCP": {
    "configuration/entityTypes/HCP/survivorshipGroups/group_test_1": [
      "ROLE_TEST_1"
    ]
  }
}
```

Additional Endpoints for Mapping

You can use the two endpoints mentioned below to add a mapping between user roles and created groups.

Use the following request to create a mapping for the specified entity type name:

```
PUT: {TenantURL}/rulesetsConfiguration/entityTypes/{entityTypeName}/survivorshipGroups
BODY:
```

```
{
  "configuration/entityTypes/HCP/survivorshipGroups/group_test_1": [
    "ROLE_TEST_1"
  ],
  "configuration/entityTypes/HCP/survivorshipGroups/group_test_2": [
```

Use the following request to create a mapping for the specified **survivorshipGroup** in entity type:

```
PUT: {TenantURL}/rulesetsConfiguration/entityTypes/{entityTypeName}/survivorshipGroups/{groupUri}

BODY:
{
  "uri": "configuration",
  "label": "Layer3Configuration",
  "description": "simple-l3.v1",
  "schemaVersion": "1",
  "referenceConfigurationURI": "configuration/_vertical/life-sciences",
  "abstract": "false",
  "entityTypes": [
    {
      "uri": "configuration/entityTypes/Organization",
      ...
      "survivorshipGroups": [
        {
          "uri": "configuration/entityTypes/Organization/survivorshipGroups/default",
          "label": "Group A",
          "description": "Use Aggregate Values",
          "default": false,
          "roles": [
            "ROLE_nPtADYqmF3qu1iy_SURVIVORSHIP"
          ],
          "mapping": [
            {
              "attribute": "configuration/entityTypes/Organization/attributes/Address",
              "survivorshipStrategy": "Aggregation"
            },
            {
              "attribute": "configuration/entityTypes/Organization/attributes/Identifiers",
              "survivorshipStrategy": "Aggregation"
            }
          ]
        }
      ]
    }
  ]
}
...
}
```

Set Cleanse Functions Configuration

Use this operation to configure cleanse functions.

 **Note:** This operation overrides the **cleanse** section in your tenant's physical configuration.

Request:

```
PUT {{applicationURL}}/tenants/{{tenantID}}/cleanse
```

Example:

```
PUT https://dev.reltio.com/reltio/tenants/hX777YXpWGBFUh/cleanse
```

Parameters

Name		Required	Description
Headers	Content-Type	Yes	Content-Type: application/json
	Authorization	Yes	Access token in format "Bearer <accessToken>". See Auth API .
Body		Yes	JSON object with Cleanse Functions specification.

Response

If the cleanse functions specification is applied successfully, then the response contains a JSON object the same specification.

```
PUT {{applicationURL}}/tenants/{{tenantID}}/cleanse
Body:
[
  {
    "cleanseFunction": "Loqate",
    "options": {
      "returnUnverifiedStatus": "true",

      "loqateDefaultCountry": "USA",
      "loqateLicensedCountry": "US;United States;USA;Canada;CA;Australia;AU"
    }
  },
  {
    "cleanseFunction": "VerificationStatusCleanser",
    "type": "S3File",
    "options": {
      "bucket": "cleanse-files",
      "path": "Sample/test.txt"
    }
  },
  {
    "cleanseFunction": "HttpStatusCleanser",
    "type": "Http",
    "options": {
      "url": "",
      "method": "GET",
      "lqtkey": "EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE",
      "p": "v+g",
      "opts": "DefaultCountry=RUS"
    }
  },
  {
    "cleanseFunction": "UppercaseCleanser",
    "type": "com.reltio.cleanser.sample.UppercaseCleanser",
    "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
  },
  {
    "cleanseFunction": "MECleanser",
    "type": "com.reltio.cleanser.sample.MECleanser",
    "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
  },
  {
    "cleanseFunction": "NPICleanser",
    "type": "com.reltio.cleanser.sample.NPICleanser",
    "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
  }
]
```

Here is a description of each function in the example. For details on our OOTB functions, see [Out-of-the-box Cleanse Functions](#).

cleanseFunction details

cleanseFunction	Description
Loqate	Integrates with the Loqate service to validate and standardize address information.
VerificationStatusCleanser	Uses a file from an S3 bucket to verify and cleanse data based on predefined rules.
HttpStatusCleanser	Simple HTTP cleanser. For more details, see topic HTTP Cleanse Function .  Note: You cannot post or send any security headers through <code>HttpStatusCleanser</code> function.
UppercaseCleanser	Converts text data to uppercase to ensure consistency.
MECleanser	A custom cleanser designed for specific cleansing requirements.
NPICleanser	A custom cleanser tailored for National Provider Identifier (NPI) data cleansing.

Example Response

```
[
  {
```



```
"cleanseFunction": "Loqate",
"options": {
  "returnUnverifiedStatus": "true",
  "loqateDefaultCountry": "USA",
  "loqateLicensedCountry": "US;United States;USA;Canada;CA;Australia;AU"
```

```
"type": "S3File",
"options": {
  "bucket": "cleanse-files",
  "path": "Sample/test.txt"
}
},
{
  "cleanseFunction": "HttpStatusCleanser",
  "type": "Http",
  "options": {
    "url": "",
    "method": "GET",
    "lqtkey": "EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE",
    "p": "v+g",
    "opts": "DefaultCountry=RUS"
  }
},
{
  "cleanseFunction": "UppercaseCleanser",
  "type": "com.reltio.cleansample.UppercaseCleanser",
  "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
},
{
  "cleanseFunction": "MECleanser",
  "type": "com.reltio.cleansample.MECleanser",
  "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
},
{
  "cleanseFunction": "NPICleanser",
  "type": "com.reltio.cleansample.NPICleanser",
  "module": "cleanse-files/Sample/UppercaseCleanserEx.jar"
}
}
]
```

Set Configuration

Use this API to set or update the tenant business configuration. If a configuration already exists, this operation overrides it.

Before creating the tenant configuration, ensure that the tenant storage configuration is created.

To retrieve or update a specific object type, see [Manage configuration by object type](#).

HTTP method and endpoint

Use the following HTTP method and endpoint path to submit the request to set the tenant configuration.

```
PUT {TenantURL}/configuration
```

You can use the **checkParallelUpdate** query parameter to prevent unintended overwrites during concurrent updates. This parameter introduces stale update validation by comparing the latest configuration state with the update request.

For more information, see [Concurrent update validation behavior](#).

Request headers

The following request headers must be included.

Headers	Required	Value
Content-Type	Yes	application/json

Request body

The request body must contain the tenant business configuration in JSON format.

Parameter	Type	Required	Description	Accepted values / Default
-----------	------	----------	-------------	---------------------------

ReltioProductsDeveloper resourcesRolesApplicationsObjectives

description	String	No	Description of the configuration.	—
schemaVersion	String	Yes	Version of the configuration schema.	—
updatedAtTime	Number	Conditional	Timestamp from the latest configuration read. Used when <code>checkParallelUpdate=true</code> .	Must match latest value
entityTypes	Array	Yes	List of entity type definitions included in the configuration.	—
entityTypes				
URI	String	Yes	Unique identifier for the entity type.	—
name	String	Yes	Name of the entity type.	—
label	String	Yes	Display label for the entity type.	—
description	String	No	Description of the entity type.	—
attributes	Array	Yes	List of attributes defined for the entity type.	—
attributes				
URI	String	Yes	Unique identifier for the attribute.	—
name	String	Yes	Name of the attribute.	—
label	String	Yes	Display label for the attribute.	—
type	String	Yes	Data type of the attribute (for example, <code>String</code>).	—

Example request

```
PUT {TenantURL}/configuration?checkParallelUpdate=true
Content-Type: application/json

{
  "label": "Test Business Model",
  "description": "Test Business Model Configuration",
  "schemaVersion": "1",
  "updatedAtTime": 1700000000000,
  "entityTypes": [
    {
      "URI": "configuration/entityTypes/Individual",
      "label": "Individual",
      "name": "Individual",
      "description": "Represents people",
      "attributes": [
        {
          "URI": "configuration/entityTypes/Individual/attributes/FirstName",
          "name": "FirstName",
          "label": "First Name",
          "type": "String"
        }
      ]
    }
  ]
}
```

Response body

--	--	--

name	String	Name of the business model configuration.
description	String	Description of the configuration.
schemaVersion	String	Version of the configuration schema.
updatedAt	Number	Timestamp of the latest successful configuration update.
entityTypes	Array	List of entity type definitions in the saved configuration.
entityTypes		
URI	String	Unique identifier for the entity type.
name	String	Name of the entity type.
label	String	Display label for the entity type.
description	String	Description of the entity type.
id	String	Generated identifier for the entity type.
attributes	Array	List of attributes defined for the entity type.
attributes		
URI	String	Unique identifier for the attribute.
name	String	Name of the attribute.
label	String	Display label for the attribute.
type	String	Data type of the attribute.
id	String	Generated identifier for the attribute.

Example response

```
{
  "label": "Test Business Model",
  "description": "Test Business Model Configuration",
  "schemaVersion": "1",
  "updatedAt": 1700000005000,
  "entityTypes": [
    {
      "URI": "configuration/entityTypes/Individual",
      "label": "Individual",
      "name": "Individual",
      "id": "1",
      "description": "Represents people",
      "attributes": [
        {
          "URI": "configuration/entityTypes/Individual/attributes/FirstName",
          "name": "FirstName",
          "label": "First Name",
          "id": "2",
          "type": "String"
        }
      ]
    }
  ]
}
```

When multiple users or systems update the tenant business configuration at the same time, updates based on outdated data can overwrite more recent changes. To prevent this, the API supports concurrent update validation using the `checkParallelUpdate` query parameter. This validation applies only to the `PUT /configuration` endpoint.

- If no business model exists, the request is treated as a first-time save and is accepted.
- If a business model exists:
 - If `updatedAtTime` is not provided, the update is accepted.
 - If `request.updatedTime` is greater than or equal to the stored `updatedAtTime`, the update is accepted and persisted.
 - If `request.updatedTime` is less than the stored `updatedAtTime`, the update is rejected with HTTP `409 (Conflict)` because the request is based on an outdated configuration state.

After every successful update, the system assigns a new `updatedAtTime` to reflect the latest configuration state.

When validation is disabled (default behavior), if `checkParallelUpdate` is not provided or is set to `false`, the system skips validation and processes the update without comparing `updatedAtTime`. In this case, the request can overwrite more recent configuration changes.

Errors

Code	Description
301	Invalid Invalid rule specification in model for rule <code>{0}</code> .
303	Business model schema cannot be parsed as XSD.
304	Failed to validate business model.
305	Failed to press business model JSON during validation.
306	Failed to process business model JSON in create configuration request.
409	Update rejected due to stale <code>updatedAtTime</code> when <code>checkParallelUpdate=true</code> .

Set Source Publish Date for Source System

Request

```
PUT{
  TenantURL
  //configuration/sources/{
    sourceName
  }/sourcePublishDate{
    "sourcePublishDate":1378187168000
  }
}
```

Updated on April 17, 2026